

=====

Spring Boot & Microservices (27 - SBMS)

=====

Trainer : Mr. Ashok

Pre-Requisites : Core Java + ADV Java + SQL

Download Course Content : <https://ashokitech.com/spring-boot-microservices-online-training>

=====

Course Content :

=====

Spring Core

Spring Boot

Spring Data JPA

Spring Web MVC

RESTFul Services

Microservices

Spring Security

Spring Cloud

Apache Kafka

Redis Cache

Cloud Deployment

Tools : Maven + Junit 5 + Logging + Jenkins Deployment + Docker deployment

Course Duration : 3 - 3.5 Months

Class Timings : 7:30 PM - 9:00 PM (IST) (Mon - Sat)

Class Mode : Everyday Live Class in Zoom

Notes: Topic wise class notes will be provided in portal (www.ashokitech.com)

Plan-1 : Course Fee : 7000 INR (Live Classes + Materials)

Plan-2 : Course Fee : 10,000 INR (Live Classes + Materials + Backup Videos - 6 Months)

Note: One telegram / whatsapp group will be created for technical discussions

=====

Programming Language Vs Technology Vs Framework

=====

=====

Q) What is a Programming Language ?

=====

=> Programming Language contains set of Programs / Set of Instructions

Ex: C , C++ , Java, C#, Python etc....

=> Every Programming Language will follow set of Syntaxes / Rules

=> Using Programming Language we can develop

Applications / Tools / Softwares / Technologies / Frameworks

=====

Q) What is Technology ?

=====

=> Technology is a software which is developed by using Programming Languages

Ex: Servlets, JSP, JDBC

=> Technologies are used to simplify our task

=> By using Technologies we can develop several types of applications

Project = Business Logic + Common Logics

=====

Q) What is a Framework ?

=====

=> To overcome the loops holes of Technologies Frameworks came into picture

=> Framework is a semi developed software

=> Framework provides common logics required for application development

Ex:

- 1) Capture form data
- 2) Validate Form Data
- 3) Create Connection Pool
- 4) DB CRUD Operations etc.....

=> Framework provides Re-Usable components

=> For example, every Java Developer should write below lines of code to perform DB operations

```
Class.forName("");
```

```
DM.getConn("");
```

```
createStatement ( )
```

```
executeQuery ( )
```

```
process ResultSet
```

```
close connection
```

Note: If we are writing same code for multiple times then it is called as Boiler Plate code / Redundent Code.

=> To avoid duplicate code / common logics in project Frameworks came into picture.

Project = Business Logics + Common Logics

=> Frameworks provides Common Logics required for the project so that we can focus only on Business Logic development.

=> Frameworks will improve developers productivity (we can do more work in less time)

Ex:

JAVA => Hibernate, Struts, Spring ...

.Net => WCF

Python => Django, Flask

Salesforce => Lightning etc

=====

Tools

=====

=> Tools are used to automate manual work

Ex:

1) Maven

2) JIRA

3) JENKINS

4) JMETER

5) POSTMAN

6) SONARQUBE etc.....

=====

Note: Frameworks will be developed by using Programming Languages only.

Core Java ==> Programming Language

JDBC + Servlets + JSP ==> Technologies

Hibernate + Struts + Spring + Spring Boot ==> Frameworks

=====

=====

Application Architecture

=====

Project : Collection of programs

Project = Frontend + Backend + Database

Frontend : Presentation Logic / User Interface Logic

Frontend Techstack :

- 1) HTML & CSS
- 2) Java Script
- 3) Bootstrap
- 4) Angular (or) React JS (or) Vue JS

Backend : Web Layer (REST APIs) + Business Layer + Integration Layer + DAO Layer

Backend Tech Stack :

- 1) Java & J2EE
- 2) Spring Core
- 3) Spring Boot
- 4) JPA
- 5) REST APIs
- 6) JSON
- 7) Microservices
- 8) Security

9) Kafka + Redis

10) Reactive Programming

Database : Persistent Store

Ex: Oracle, MySQL, Postgres, Mongo DB etc...

Tools : Maven + Git Hub + Log4J + JUnit + Jenkins + SonarQube + JIRA + Docker + Kubernetes

Cloud Platforms : AWS / Azure / GCP

=====

Architecture Types

=====

1) Monolith Architecture

2) Microservices Architecture

=====

Framework

=====

=> Semi Developed software which provides common logics required for projects development

=> Frameworks will help the developers to implement more functionality in less time

=> When we use framework to develop the project, we can focus only on business logic.

=====

Types of Frameworks

=====

1) Frontend Frameworks : To develop user interface in the project.

Ex: Angular

2) Web Frameworks : To develop web layer in the project

Ex: Struts (Outdated)

3) ORM Frameworks : To develop persistence layer in the project.

Ex: Hibernate

=> By using Struts we can develop only Web Layer in the Project (Controllers)

=> By using Hibernate we can develop only Data Access Layer (Persistence Layer)

Note: To overcome the problems of Struts framework, Spring Framework came into market.

=> Spring Framework is called as Application Development Framework

=> By using Spring framework we can develop end to end application

=> Spring is free & open source framework

=> Spring Framework developed in Modular Fashion

Note: Spring framework means collection of modules

=====

Spring Modules

=====

- 1) Spring Core
- 2) Spring Context
- 3) Spring JDBC
- 4) Spring ORM
- 5) Spring AOP
- 6) Spring Web MVC
- 7) Spring Security
- 8) Spring Social
- 9) Spring Batch
- 10) Spring Data JPA
- 11) Spring REST
- 12) Spring Cloud

Note: Spring is very flexible framework. It will not force to use all modules. Based on requirement we can pickup particular module and we can use it.

=> Spring is versatile framework (Easily it can be integrated with other frameworks)

=> The current version of Spring framework is 6.0

=> Spring framework is under license of VM Ware Tanza...

URL : www.spring.io

=====

1) Spring Core : It is base module in the spring framework.

=> Spring Core Module providing fundamental concepts of Spring Framework

- 1) IOC Container (Inversion Of Control)
- 2) Dependency Injection
- 3) Bean Life cycle
- 4) Bean Scopes
- 5) Autowiring etc...

2) Spring Context : It will deal with configurations required for our Spring Applications.

3) Spring AOP : Aspect Oriented Programming

=> AOP is used to separate business logics & Secondary logics in the project

Ex: Security, Logging, Tx, Auditing, Exception Handling...

Note: If we combine business logics & secondary logics then we will face maintenance issues of our project.

4) Spring JDBC : Spring JDBC is used to simplify Database Communication logic

=> In java jdbc we need to write boiler plate code (repeated code) like below in several classes

- => Load driver
- => Get connection
- => Create Statement
- => Execute Query
- => Close connection

=> Using Spring JDBC we can directly execute query the remaining part Spring JDBC will take care

5) Spring Web MVC : It is used to develop both Web Applications & Distributed Applications

=> Web Applications (C 2 B)

Ex: Gmail.com, facebook.com etc...

=> Distributed Applications (B 2 B) / Web Services or RESTFul Service

Ex:

IRCTC ---- MakeMyTrip

Passport --- AADHAR app

6) Spring ORM (Object Relational Mapping)

=> Spring Framework having integration with ORM frameworks

Ex: Spring ORM , Spring Data JPA etc....

Note: JDBC will represent data in text format where as Hibernate ORM will represent data in Objects format.

7) Spring Security

=> Security is very crucial for every application

=> Using Spring Security We can implement Authentication & Authorization

=> Spring Security with OAuth2.0

=> Spring Security with JWT (JSON Web Tokens)

8) Spring Batch : Batch means bulk operation

=> Reading data from Excel and store it into database table

=> Sending Monthly statements to customers in email

=> Sending Reminders to customers as Bulk SMS

9) Spring Cloud : It provides some common tools to quickly build distributed systems.

=> It provides service registry to register all our microservices at one place

=> It provides API Gateway to have single entry point for all our apis

=> Load Balancer

=> Monitoring

=> Circuit Breaker (Fault Tolerant Systems / Resilience)

=> Distributed Messaging

=> Routing

10) Spring Test : It provides Unit Test framework

=====

Spring Core : It is all about Managing dependencies among the classes with loosely coupling

=====

=> In project we will develop several classes. All those classes we can categorize into 3 types

1) POJO

2) Java Bean

3) Component

=====

What is Pojo (Plain Old Java Object)

=====

=> Any Java class which can be compiled by using only JDK software is called as POJO class.

Ex-1 : Below class is valid POJO

```
class Demo1 {  
    int id;  
    String name;  
}
```

Ex-2 : Below class is valid POJO

```
class Demo2 extends Thread {  
    int id;  
    String name;  
}
```

Ex-3 : Below class is valid POJO

```
class Demo3 implements Runnable {  
    // run method  
}
```

Ex-4 : Below class is not POJO because Servlet is part of JEE

class Demo4 implements Servlet {

 // run method

}

=====

What is Java Bean ?

=====

=> Any java class which follows bean specification rules is called as Java Bean.

- 1) Class should implement serializable interface
- 2) Class should have private data members (variables)
- 3) Every private variable should have public setter & public getter method
- 4) Class should have zero-param constructor

Note : Bean classes are used to write business logic and to store and retrieve data

=====

What is Component ?

=====

=> The java classes which contains business logic is called as Component classes

Ex: Controllers, Services , Dao classes

=> Controller classes will have logic to deal with Request & Response

=> Service classes will have business logic of our project

Ex: Generate OTP, Send OTP, Send Email, Encrypt & Decrypt PWD etc...

=> DAO classes will contain the logic to communicate with Database

=====

=> In a project we will develop multiple classes and those classes will be dependent on other classes.

Ex:

=> Controller class will call service class methods

=> Service class will call Dao class methods

=> In Java one class can talk to another class in 2 ways

- 1) Inheritance (IS - A)
- 2) Composition (HAS - A)

=====

Car & Engine Example

=====

```
class Engine {  
    void start ( ) {  
        // logic  
    }  
}  
  
class Car {  
    void drive( ) {  
        // star the engine  
        // drive the car  
    }  
}
```

=> If we want to drive the car then we need to start the Engine that means Car class drive () method should call Engine class start () method.

Q) In how many ways Car class can call Engine class method ?

=> in 2 ways

- 1) Inheritance
- 2) Composition

===== IS-A approach =====

```
package in.ashokit;
```

```
public class Engine {
```

```
    public int start ( ) {
```

```
        // logic
```

```
        return 1;
```

```
    }
```

```
}
```

```
package in.ashokit;
```

```
public class Car extends Engine {
```

```
    public void drive() {
```

```
        // start the engine
```

```
        int start = super.start();
```

```
        if (start >= 1) {
```

```
            System.out.println("Journey Started");
```

```
        }
```

```
        // start the journey
```

```
    }
```

```
}
```

===== HAS - A Relation =====

```
public class Car {
```

```
    public void drive() {
```

```
        Engine eng = new Engine ( );
```

```
        int start = eng.start();
```

```
        if (start >= 1) {
```

```
            System.out.println("Journey Started");
```

```
        }
```

```
        // start the journey
```

```
    }
```

```
}
```

Note: Tomorrow, if Engine class constructor modified then our Car class will be effected.

=====

Note: If we use any approach from above then Car class will become tightly coupled with Engine class. That is not recommended.

Always we need to develop our classes with Loosely Coupling
#####

=> Loosely coupling means without creating Object and without Inheriting properties we should be able to access one class method in another class.

=> If we make any changes in Engine class then Car class shouldn't be effected then we can say our classes are loosely coupled.

To develop classes with loosely coupling we need to use Interfaces

package in.ashokit;

```
public class Car {  
    private IEngine eng;  
    public Car(IEngine eng) {  
        this.eng = eng;  
    }  
    public void drive() {  
        int start = eng.start();  
        if (start >= 1) {  
            System.out.println("Journey Started...");  
        } else {  
            System.out.println("Engine in trouble...");  
        }  
    }  
}
```

```

-----
package in.ashokit;

public class Main {

    public static void main(String[] args) {

        Car car = new Car (new PetrolEngine());

        car.drive();

    }

}

```

What is Dependency Injection ?

=> The process of injecting one class object into another class is called as 'Dependency Injection'.

=> We can perform Dependency Injection in 3 ways

- 1) Setter Injection
- 2) Constructor Injection
- 3) Field Injection

----- Example to Understand Dependency Injection -----

```

public class Car {

    private IEngine eng ;

    public void setEng(IEngine eng) {

        this.eng = eng;

    }

    public void drive() {

        int start = eng.start();

        if (start >= 1) {

            System.out.println("Journey Started...");

        } else {

            System.out.println("Engine in trouble...");

        }

    }

}

```

=> In the above program 'Car' class is dependent on 'Engine' object that means 'Engine' class object should be injected into 'Car' class.

Note: Car is dependent on Engine

=====

Setter Injection (SI)

=====

=> Setter Injection means, Injecting dependent object into target object using target class setter method.

```
public class Car {  
    private IEngine eng;  
    public void setEng(IEngine eng) {  
        this.eng = eng;  
    }  
    public void drive() {  
        int start = eng.start();  
        // logic  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // creating target obj  
        Car car = new Car();  
        // injecting dependent obj into target thru setter method (Setter Injection - SI)  
        car.setEng(new PetrolEngine());  
        car.drive();  
    }  
}
```

=====

Constructor Injection (CI)

=====

=> Constructor Injection means, Injecting dependent object into target object using target class constructor.

```
public class Car {  
    private IEngine eng;  
    public Car (IEngine eng) {  
        this.eng = eng;  
    }  
    public void drive() {  
        int start = eng.start();  
        // logic  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // creating target obj ( Constructor Injection )  
        Car car = new Car(new DieselEngine());  
        car.drive();  
    }  
}
```

=====

Q) Can we perform both SI & CI for single variable ?

Yes, but Setter Injection will override Constructor Injection value.

```
public class Main {  
    public static void main(String[] args) {  
        // creating target obj ( Constructor Injection - CI )  
        Car car = new Car(new DieselEngine());
```



```

        // Setter Injection - SI
        car.setEng(new PetrolEngine());
        car.drive();
    }
}

```

=====

Field Injection - FI

=====

=> Field Injection means, injecting depending object into target class using target class variable is called as Field Injection.

```

public class Car {
    private IEngine eng;

    public void drive() {
        int start = eng.start();
        if (start >= 1) {
            System.out.println("Journey Started...");
        } else {
            System.out.println("Engine in trouble...");
        }
    }
}

```

Note: We can access private variables outside of the class using Reflection API like below
#####

```

public class Main {
    public static void main(String[] args) throws Exception {
        Class<?> clz = Class.forName("in.ashokit.Car");
        Object object = clz.newInstance();
        Car carObj = (Car) object;
        Field engField = clz.getDeclaredField("eng");
        engField.setAccessible(true);
    }
}

```

```
        // Injecting value to variable
        engField.set(carObj, new PetrolEngine());
        carObj.drive();
    }
}
```

=====

```
List l = new ArrayList ( ) ; // valid
List l = new LinkedList ( ) ; // valid
List l = new Vector ( ) ; // valid
IEngine eng = new DieselEngine ( ) ; // valid
IEngine eng = new PetrolEngine ( ) ; // valid
```

=====

IOC Container

=====

-> IoC stands for Inversion Of Control.

-> IoC is responsible for Dependency Injection in Spring Applications.

-> Dependency Injection means creating and injecting dependent bean objects into target bean classes.

Note: IoC container will manage life cycle of Spring Beans.

Note : We need to provide " Java classes + Bean Configuration " as input for IOC then IOC will perform DI and provides Spring Beans which are ready to use.

=====

What is Spring Bean ?

=====

-> Any Java class whose lifecycle (creation to destruction) is managed by IOC is called as Spring Bean.

-> We can represent Java class as Spring Bean in 2 ways

1) XML Approach

Ex: <bean id ="id1" class = "pkg.ClassName" />

2) Annotation Approach (Recommended)

Ex: @Component, @Service, @Repository etc....

Note: In Spring we can use both XML & Annotation approaches. SpringBoot will support only Annotations (no xmls)

=====

How to Start IoC in Spring ?

=====

1) BeanFactory (Outdated)

2) ApplicationContext (recommended)

Ex: ApplicationContext context = new ClassPathXmlApplicationContext(String configFile);

Note: Bean Configuration file contains Bean Definitions

(target class, dependent class, Dependency Injection type)

Note: When IoC container started it will read bean definitions from Bean Configuration File
and it will perform Dependency Injection.

=====

First Application Development using Spring Core Module

=====

Pre-Requisites : JDK 1.8v, STS IDE

- 1) Create Maven Project in IDE
- 2) Add 'Spring Context' Dependency in pom.xml file (search here : www.mvnrepository.com)

```
<dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.2.22.RELEASE</version>
    </dependency>
</dependencies>
```

- 3) Create Required Java classes (Ex: IEngine, PetrolEngine, DieselEngine and Car)
- 4) Create Bean Configuration File and configure Bean Definitions

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="petrolEng" class="in.ashokit.beans.PetrolEngine" />
    <bean id="car" class="in.ashokit.beans.Car">
        <property name="eng" ref="petrolEng" />
    </bean>
</beans>
```

Note: Here <property/> tag represents setter injection.

5) Create Main class and start IOC Container to test the application.

```
public class App {  
    public static void main(String[] args) {  
        // starting iOC container  
        ApplicationContext context = new ClassPathXmlApplicationContext("Beans.xml");  
        // getting bean object from IOC  
        Car car = context.getBean(Car.class);  
        car.drive();  
    }  
}
```

Difference Between BeanFactory & ApplicationContext

=> BeanFactory interface having - XmlBeanFactory as implementation class

```
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("Beans.xml"));
```

=> ApplicationContext interface having - ClassPathXmlApplicationContext as implementation class

```
ApplicationContext context = new ClassPathXmlApplicationContext("Beans.xml");
```

=> BeanFactory will follow Lazy Loading concept that means when we request then only it will create Bean object.

=> ApplicationContext will follow Eager Loading for Singleton Beans. For Prototype beans it will also follow Lazy Loading

-----**Note: Spring Bean default scope is Singleton**-----

Eager Loading means creating objects for Spring Bean when IoC starts

Lazy Loading means creating objects for Spring Bean when we call getBean () method.

Note: XmlBeanFactory is deprecated that means it may not be available in future versions of Spring.

Note: It is recommended to create IoC using Application Context.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("===== BeanFactory Started =====");  
        BeanFactory factory = new XmlBeanFactory(new ClassPathResource("Beans.xml"));  
        System.out.println("===== ApplicationContext Started =====");  
        ApplicationContext context = new ClassPathXmlApplicationContext("Beans.xml");  
        Car bean = context.getBean(Car.class);  
        bean.drive();  
    }  
}
```

How to differentiate Setter Injection & Construction Injection in Bean Config File ?

=> <property /> tag represents setter injection

=> <constructor-arg /> tag represents constructor injection

```
<bean id="petrolEng" class="in.ashokit.beans.PetrolEngine" />  
<bean id="dieselEng" class="in.ashokit.beans.DieselEngine" />  
<bean id="car" class="in.ashokit.beans.Car">  
    <property name="eng" ref="dieselEng" />  
    <constructor-arg name="eng" ref="petrolEng" />  
</bean>
```

Note: When we perform both SI & CI then SI will override CI value.

=====

Bean Scopes

=====

=> Scope represents how many objects should be created for a Spring Bean

=> In Spring framework we have below scopes

- 1) singleton (default scope)
- 2) prototype
- 3) request
- 4) session

=> To represent bean scope we will use "scope" attribute

```
<bean id="id" class="pkg.classname" scope = "singleton | prototype | request | session " />
```

-> Singleton scope means only one object will be created for the class in IOC Container. This is default scope of spring bean.

-> Prototype scope means every time new object will be created.

Note: request & session scopes are related to Spring Web MVC Module.

=====

Why Spring Bean is by default Singleton ?

=====

=> To save memory of JVM spring team made singleton as default scope for the spring beans.

Ex: Rest Controllers, Controllers, Services and DAOs will be considered as Singleton in the project

Ex: TicketGenerator class is used to generate new Ticket for every customer

TicketGenerator ----> Singleton bean

Ticket -----> Prototye Bean

=====

Autowiring

=====

=> We can inject dependent bean into target in 2 ways

- 1) Manual Wiring

2) Autowiring

-> Manual wiring means programmer will inject dependent object into target object using
<property/> tag or <constructor-arg/> tag using 'ref' attribute.

```
<bean id="dieselEng" class="in.ashokit.beans.DieselEngine" />
<bean id="car" class="in.ashokit.beans.Car">
    <property name="eng" ref="dieselEng" />
</bean>
```

-> Autowiring means IoC container will identify dependent bean and it will inject into target bean
(we no need to use any ref attribute in bean configuration file)

-> Autowiring will work based on below modes

- 1) byName
- 2) byType
- 3) constructor
- 4) no

Note: Autowiring will not work by default, We have to enable autowiring on target bean like below.

```
<bean id="id" class="pkg.Classname" auto-wire="byName | byType | constructor | no " />
```

=====

byName

=====

=> byName means IoC will identify dependent bean object based on bean id or bean name.

```
public class Car {
    private IEngine eng;
    public void setEng(IEngine eng) {
        System.out.println("setEng ( ) method called....");
        this.eng = eng;
    }
}
```



```

    public void drive() {
        int status = eng.start();
        if (status >= 1) {
            System.out.println("Journey Started..");
        } else {
            System.out.println("Engine Trouble");
        }
    }
}

```

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="eng1" class="in.ashokit.beans.PetrolEngine" />
    <bean id="eng" class="in.ashokit.beans.DieselEngine" />
    <bean id="car" class="in.ashokit.beans.Car" autowire="byName"/>
</beans>

```

Note: In the above example Car class variable name is matched with 'DieselENGINE' bean id hence DieselEngine obj will be injected into Car.

=====

byType

=====

=> byType means IoC will identify dependent bean object based on data type of the variable in Target class.

```
private IEngine eng;  =====> data type of eng is IEngine which is an interface
```

=> If one interface having 2 implementations then there is a chance of getting Ambiguity problem. To overcome that we need to use 'autowire-candidate' attribute.

autowire-candidate="false" ====> Not Eligible for Autowiring

autowire-candidate="true" ====> Eligible for Autowiring

Note: As an alternate for "autowire-candidate=true" we can use "primary=true" to consider bean for Autowiring.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="eng2" class="in.ashokit.beans.PetrolEngine" autowire-candidate="false"/>
    <bean id="eng1" class="in.ashokit.beans.DieselEngine" autowire-candidate="true"/>
    <bean id="car" class="in.ashokit.beans.Car" autowire="byType" />
</beans>
```

(or)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="eng2" class="in.ashokit.beans.PetrolEngine" primary="true" />
    <bean id="eng1" class="in.ashokit.beans.DieselEngine" />
    <bean id="car" class="in.ashokit.beans.Car" autowire="constructor" />
</beans>
```

=====

constructor

=====

=> It is used to perform Autowiring by calling target class constructor

package in.ashokit.beans;

```
public class Car {  
    private IEngine eng;  
    public Car(IEngine eng) {  
        this.eng = eng;  
    }  
    public void drive() {  
        int status = eng.start();  
        if (status >= 1) {  
            System.out.println("Journey Started..");  
        } else {  
            System.out.println("Engine Trouble");  
        }  
    }  
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="http://www.springframework.org/schema/beans
```

```
    http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
    <bean id="eng2" class="in.ashokit.beans.PetrolEngine" autowire-candidate="false"/>
```

```
    <bean id="eng1" class="in.ashokit.beans.DieselEngine" autowire-candidate="true"/>
```

```
    <bean id="car" class="in.ashokit.beans.Car" autowire="constructor" />
```

```
</beans>
```

=====

Note: Autowiring is applicable for Reference Type variable (not applicable for primitive types)

=====

Spring Bean Life Cycle

=====

=> Life cycle means starting to ending or birth to death

- > Thread Life Cycle
- > Servlet Life Cycle
- > JSP Life cycle
- > Spring Bean Life Cycle

=> Spring Bean object creation and object destruction will be taken care by IOC container.

=> Spring Bean Life Cycle will be managed by loc Container.

=> We can perform some operations using Bean Life Cycle Methods

init () -----> initialization logic

destroy () -----> destruction logic

=> Spring Bean Life Cycle methods we can execute in 3 ways

- 1) XML Approach (Declarative)
- 2) Programmatic approach
- 3) Annotations

===== **Bean Life Cycle using XML Approach** =====

```
<bean id="motor" class="in.ashokit.beans.Motor"
      init-method="start"
      destroy-method="stop"/>
```

init-method ==> It represents the method which should be called after bean obj created

destroy-method ==> It represents the method which should be called when bean obj removing from IoC

```
package in.ashokit.beans;
```

```
public class Motor {  
    public Motor() {  
        System.out.println("Motor :: Constructor");  
    }  
    public void start() {  
        System.out.println("Motor started....");  
    }  
    public void doWork() {  
        System.out.println("Motor Pulling Water...");  
    }  
    public void stop() {  
        System.out.println("Motor stopped.....");  
    }  
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
    <bean id="motor" class="in.ashokit.beans.Motor"
```

```
        init-method="start" destroy-method="stop" />
```

```
</beans>
```

===== Bean Life Cycle using Programmatic Approach =====

=> We need to implement predefined interfaces provided by Spring Framework

1) InitializingBean ---> afterPropertiesSet ()

2) DisposableBean ---> destroy ()

```
public class Motor implements InitializingBean, DisposableBean {  
    public Motor() {  
        System.out.println("Motor :: Constructor");  
    }  
    public void afterPropertiesSet() throws Exception {  
        System.out.println("motor started.....");  
    }  
    public void doWork() {  
        System.out.println("Motor Pulling Water...");  
    }  
    public void destroy() throws Exception {  
        System.out.println("motor stopped.....");  
    }  
}
```

===== Bean Life Cycle using Annotation Approach =====

@PostConstruct -----> It represents init method

@PreDestroy -----> It represents destroy method

```
public class Motor {  
    public Motor() {  
        System.out.println("Motor :: Constructor");  
    }  
    @PostConstruct  
    public void m1() throws Exception {  
        System.out.println("motor started.....");  
    }  
}
```

```

    public void doWork() {
        System.out.println("Motor Pulling Water...");
    }

    @PreDestroy
    public void m2() throws Exception {
        System.out.println("motor stopped.....");
    }
}

```

=====

- 1) What is Spring Bean
- 2) How to represent Java class as Spring Bean
- 3) What is IoC Container
- 4) What is Bean Configuration File
- 5) How to start IOC Container
- 6) First Application Development using Spring Core Module
- 7) What is Dependency Injection (SI , CI & FI)
- 8) BeanFactory vs ApplicationContext
- 9) Lazy loading vs Eager Loading
- 10) Bean Scopes (Singleton & Prototype)
- 11) Autowiring & Modes
- 12) Bean Life Cycle Methods (XML, Programmatic & Annotations)

=====

Spring Annotations

=====

Annotation ==> Represent Metadata

=> Annotations Introduced in Java 1.5v

=> Annotations are alternate for xml configurations

=> In Spring Framework we have several annotations

1) **@Configuration** : To represent java class as Configuration class

2) **@ComponentScan** : To identify Spring Bean classes available in the project based on "basePackageNames"

3) **@Component** : To represent java class as Spring Bean.

4) **@Service**: To represent java class as Spring Bean (Business layer classes)

5) **@Repository**: To represent java class as Spring Bean (Persistence layer classes)

6) **@Scope** : To represent bean scope

7) **@Autowired** : To perform dependency injection

8) **@Qualifier** : To perform autowiring based on byName

9) **@Primary** : To represent primary bean for Autowiring

10) **@Bean** : To call the method which returns bean object

=====

basePackage = in.ashokit

in.ashokit.beans -----> will be scanned

in.ashokit.utils -----> will be scanned

in.ashokit.dao -----> will be scanned

com.ashokit.service ---> will not be scanned

=====

Spring Boot

=====

=> Spring Boot is one approach to develop Spring Based Applications with less configurations.

Spring Boot = Spring Framework - Xml Configurations

=> Spring Boot is not replacement for Spring Framework. Boot developed on top of Spring Framework

Note: All Spring Framework concepts can be used in Spring Boot also.

=====

Spring Boot - Advantages

=====

1) Less Configuration & No Xmls Configurations

2) Pom Starters to simplify dependencies configuration

Ex: web-starter, jpa-starter, security-starter, mail-starter

3) Auto Configuration

4) Embedded Servers (Ex: Tomcat, Jetty, Netty)

5) Actuators (Production Ready Features)

=====

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run".

=====

Spring Boot 1.0 released in 2014

Current version of Spring Boot is 3.x ==> Nov-2022

Note: Java 17 is mandatory to work with Spring Boot 3.x version

=====

Spring Boot Application Creation

=====

=> We can create boot application in 2 ways

1) Initializr website (start.spring.io)

2) Spring Starter Project in IDE

Note: If we try to create boot application using IDE then internally IDE will communicate with Intializr website to create the project.

Note: Internet Connection is mandatory for system to create Spring Boot Application.

=====

Options to choose While creating boot application

=====

Build Tool : Maven / Gradle

Language : Java / Groovy / Kotlin

groupId : It represents company name / Project Name

Ex: in.ashokit

artifactId : It represents project name / module name

Ex: MyFirstApp / UserManagementApp / TicketBookingApp

version : 0.0.1 - SNAPSHOT

SNAPSHOT - Project under development

RELEASE / FINAL - Project development completed

packageName : It represents base package in the project

Ex: in.ashokit

=====

Spring Boot Application Folder Structure

=====

src/main/java : To keep our project source code

- Application.java : It is start class of the spring boot (main class)

src/main/resources : To keep project configuration files

- application.properties / yml

src/test/java : To keep junit code (unit testing)

- ApplicationTest.java

src/test/resources : Unit testing related config files goes here

Maven dependencies : Downloaded jars will be available here

pom.xml : Maven configuration file (dependencies)

=====

Spring Boot Start Class

=====

@SpringBootApplication

```
public class Application {  
    public static void main(String[] args) {  
        SpringApplication.run(Application.class, args);  
    }  
}
```

=> It is entrypoint for boot application execution

=> **@SpringBootApplication annotation is equal to below 3 annotations**

@SpringBootConfiguration

@EnableAutoConfiguration

@ComponentScan

=> Spring Boot start class will act as Configuration class because of @SpringBootConfiguration annotation

=> In Spring boot application auto configuration feature will be available because of @EnableAutoConfiguration annotation

=> Component Scan will be performed in spring boot because of @ComponentScan annotation

Note: Package Naming convention will play important role in component scanning

basePackage : in.ashokit

in.ashokit.service -- will be scanned

in.ashokit.dato -- will be scanned

com.ashokit.util ---- will not get scanned

=====

What is SpringApplication.run () method in Application class

=====

=> Spring Boot start class main () method will call SpringApplication.run () method.

=> SpringApplication.run () is entry point for boot application execution. This run () method will return reference of IOC Container.

Note: SpringApplication is a predefined class and it will identify what type of application we have created based on dependencies added in pom.xml file.

=> If we add "spring-boot-starter" ==> standalone app

--> for standalone app, It will use "AnnotationConfigApplicationContext" class to start IoC container

=> If we add "spring-boot-starter-web" ==> Web application

--> For web apps, "AnnotationConfigServletWebServerApplicationContext" class will be used to start IoC container

=> If we add "spring-boot-starter-webflux" ==> Reactive Application

--> For reactive app, "AnnotationConfigReactiveWebServerApplicationContext" class will be used to start IoC container

Note: Based on Type of our application, It will start IOC container.

=> run () method will print banner on the console (i.e spring logo).

=> run () method will start IoC container

=> run () method will call runners

=> run () method will return context of IoC container

ConfigurableApplicationContext context = SpringApplication.run(Application.class, args);

=====

What is Banner in Spring Boot ?

=====

=> When we run boot application, it will print spring logo on the console that is called as Banner in Spring Boot.

=> Spring Boot banner is having 3 modes

- 1) Off
- 2) Log
- 3) Console (default)

=> We can set banner mode using below property in "application.properties" file

spring.main.banner-mode=off

=> We can customize banner text in spring boot application by creating "banner.txt" file in src/main/resources folder

URLToGenerateASCIIText :
<https://patorjk.com/software/taag/#p=display&f=Graffiti&t=Ashok%20IT>

=====

Runners in Spring Boot

=====

=> Runners are used to execute the logic only once when boot application got started

=> SpringApplication.run () method will call runners

=> In Spring Boot we have 2 types Runners

- 1) ApplicationRunner (I)
- 2) CommandLineRunner (I)

@Component

```
public class MyAppRunner implements ApplicationRunner {  
    @Override  
    public void run(ApplicationArguments args) throws Exception {  
        System.out.println("AppRunner :: run ( ) method executed.....");  
    }  
}
```

@Component

```
public class MyCmdRunner implements CommandLineRunner{  
    @Override  
    public void run(String... args) throws Exception {  
        System.out.println("MyCmdRunner :: run ( ) method called...");  
    }  
}
```

=====

Runners Usecases

=====

1) Loading data from db to cache memory

=====

application.properties file & application.yml

=====

=> application.properties & application.yml files are used to manage configuration properties in the application

Ex: data source properties, smtp properties & application messages etc...

=> properties file is used to configure properties in linear structure

Ex:

```
spring.datasource.username=ashokit  
spring.datasource.password=ashokit@123  
spring.datasource.url=jdbc://mysql:localhost:3306/db  
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
```

=> YML / YAML file is used to configure properties in hierarichal structure

Ex:

spring:

datasource:

url: null

username: null

password: null

```
driver-class-name: null

main:

  banner-mode: false

application:

  name: app
```

=> Properties files are used only in java language

=> YML files used by several programming languages

Note: Indent spaces are very important in YML files.

=> In Properties, for every profile separate properties file should be created.

```
application_dev.properties
application_qa.properties
application_prod.properties
```

=> In YML, multiple profiles can be configured in single YML file (three --- will be used to separate profile)

Note: Spring Boot supports both .properties and .yaml.

Note: Recommended to use .yaml files

=====

Spring Boot - Summary

=====

- 1) What is Spring Boot
- 2) Spring Boot Version History
- 3) Advantages of Spring Boot
- 4) Spring Boot Application Creation
- 5) Spring Boot Application Folder Structure
- 6) What is Start Class

- 7) What is @SpringBootApplication
- 8) What is Component Scanning
- 9) What is base package naming convention ?
- 10) What is SpringApplication.run () method in Spring Boot ?
- 11) How IoC container will be created in Spring Boot ?
- 12) Which class will be used to start IoC in spring boot ?
- 13) What is the return type of SpringApplication.run () method ?
- 14) What is Banner in Spring Boot ? Can we customize that ?
- 15) What is Runner in Spring Boot ?
- 16) What is application.properties file in Spring Boot ?
- 17) What is Dependency Injection ?
- 18) What type of DI we can perform in Spring Boot ?
- 19) What is Bean Scope ?
- 20) What is Autowiring ?
- 21) What are the modes available for Autowiring ?
- 22) What is Bean Lifecycle ?
- 23) How to represent java class as Spring Bean ?
- 24) What is the difference between @Component & @Bean annotations ?
- 25) Which dependency injection is better (SI or CI or FI) ?
- 26) @Component vs @Service vs @Repository annotations ?
- 27) What is @Value annotation ?
- 28) application.properties vs application.yml
- 29) What is @Primary annotation ?

=====

Spring Data JPA

=====

=> Application contains several Layers

- 1) Presentation Layer (JSP / Thymeleaf / Angular / React JS / Vue JS)
- 2) Web Layer (Servlets / Struts / Spring Web MVC)
- 3) Persistence Layer (JDBC / Spring JDBC / Spring ORM / Spring Data JPA)

=> Spring Data JPA is used to develop Persistence layer in the application.

=> Spring Data JPA providing ready made methods to perform CRUD operation in DB tables.

=> Data JPA providing ready made methods using interfaces like below

- 1) CrudRepository (I)
- 2) JpaRepository (I)

Note : JpaRepository = CrudRepository + Pagination Methods + Sorting Methods

=====

Spring Data JPA Terminology

=====

1) Data Source Object : It represents Database Connections

Note: Data Source properties we can configure in "application.properties" or "application.yml" file

2) Entity Class : The class which is mapped with database table

@Entity

@Table

@Id

@Column

3) Repository interface : For every Table we will create one repository interface to perform Crud Operations

```
public interface StudentRepository extends CrudRepository<Student, Integer>{
```

```
}
```

Note: For our Repository interface, implementation will be provided in the runtime using Proxy Class.

Note: By using StudentRepository we can perform CRUD operations in STUDENT_TBL

4) Repository methods : Ready made methods provided by Data JPA to perform CRUD operations

1) save (Entity)

2) saveAll (Iterable<Entity> i)

Note: Above two methods are called as "UPSERT" methods (UPDATE + INSERT)

3) findById (ID id)

4) findAllById (Iterable<ID> ids)

5) findAll ()

6) count ()

7) existById (ID id)

8) deleteById (ID id)

9) deleteAllById (Iterable<ID> ids)

10) deleteAll ()

5) ORM Properties : To automate some configurations

1) auto_ddl : Dynamic Schema Generation

2) show_sql : Display generated queries on the console

=====

First Application Development Using Spring Data JPA

=====

1) Create Spring Starter Project with below dependencies

a) springboot-starter-data-jpa

b) mysql-driver

2) Create entity class and map with DB table using annotations

3) Create Repository interface to perform CRUD operations

4) Configure Data Source properties in application.yml file

5) Run the application and test the functionality

=====Student.java=====

```
@Entity
@Table(name = "STUDENT_DTLS") // optional
public class Student {
    @Id
    @Column(name="student_id") // optional
    private Integer id;
    @Column(name="student_name")
    private String name;
    @Column(name="student_rank")
    private Long rank;
    @Column(name="student_gender")
    private String gender;
    //setters & getters
}
```

=====StudentRepository.java=====

```
package in.ashokit.repository;
import org.springframework.data.repository.CrudRepository;
import in.ashokit.entity.Student;
//@Repository
public interface StudentRepository extends CrudRepository<Student, Integer>{
}
```

=====application.yml file=====

```
spring:
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    password: AshokIT@123
    url: jdbc:mysql://localhost:3306/sbms27
    username: ashokit
```

jpa:

hibernate:

ddl-auto: update

show-sql: true

=====Start Class=====

```
package in.ashokit;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.context.ConfigurableApplicationContext;
```

```
import in.ashokit.entity.Student;
```

```
import in.ashokit.repository.StudentRepository;
```

```
@SpringBootApplication
```

```
public class Application {
```

```
    public static void main(String[] args) {
```

```
        ConfigurableApplicationContext context = SpringApplication.run(Application.class,  
args);
```

```
        StudentRepository studentRepo = context.getBean(StudentRepository.class);
```

```
        Student s = new Student();
```

```
        s.setId(103);
```

```
        s.setName("Ashok");
```

```
        s.setRank(1133l);
```

```
        s.setGender("Male");
```

```
        studentRepo.save(s);
```

```
        System.out.println("Record Inserted...");
```

```
        Student s1 = new Student(103, "Orlen", 90l, "Male");
```

```
        Student s2 = new Student(104, "Cathy", 100l, "FeMale");
```

```
        Student s3 = new Student(105, "Buttler", 70l, "Male");
```

```
        Student s4 = new Student(106, "Smitha", 60l, "FeMale");
```

```
        List<Student> students = Arrays.asList(s1, s2, s3, s4);
```

```

studentRepo.saveAll(students);

Optional<Student> optional = studentRepo.findById(101);
if(optional.isPresent()) {
    System.out.println(optional.get());
}

Iterable<Student> findAll = studentRepo.findAll();

// findAll.forEach(System.out::println);

findAll.forEach(s -> System.out.println(s));

if(studentRepo.existsById(503)) {
    studentRepo.deleteById(503);
    System.out.println("Record Deleted....");
}else {
    System.out.println("No Record Present...");
}
}
}

```

=====

findByXXX methods in Data JPA

=====

=> By using findByXXX () methods we can retrieve the data based on non - primary key columns also

=> When we write findByXXX method , JPA will construct query based on method name

Note: Method Naming convention is very important for findByXXX methods

=> Using findBy methods we can perform select operations only (retrieval). INSERT / UPDATE / DELETE operations we can't do using findBy methods.

Note : In findBy method syntax we will use entity variable names.

=====Entity Class=====

```
@Entity
@Table(name = "STUDENT_DTLS") // optional
public class Student {

    @Id
    @Column(name = "student_id") // optional
    private Integer id;

    @Column(name = "student_name")
    private String name;

    @Column(name = "student_rank")
    private Long rank;

    @Column(name = "student_gender")
    private String gender;

    // setters & getters

}
```

=====Repository Interface=====

```
public interface StudentRepository extends CrudRepository<Student, Integer>{

    // select * from student_dtls where student_gender=:gender
    public List<Student> findByGender(String gender);

    // select * from student_dtls where student_gender is null
    public List<Student> findByGenderIsNull();

}
```

```

// select * from student_dtls where student_rank >= : rank
public List<Student> findByRankGreaterThanOrEqualTo(Long rank);

// select * from student_dtls where student_rank <= : rank
public List<Student> findByRankLessThanOrEqualTo(Long rank);

// male students who are having rank >=100 ;
// select * from student_dtls where student_gender=? and student_rank >= :rank

public List<Student> findByGenderAndRankGreaterThanOrEqualTo(String gender, Long rank);

}

```

=====

Custom Queries

=====

- => We can execute Custom Queries also in JPA (our own queries)
- => To execute custom queries we will use @Query annotation
- => @Query will support for executing both HQL queries & Native SQL queries also.

HQL : Hibernate Query Language (Database Independent Queries)

- => In HQL, we will use Entity class name & Entity class variables to write query
- => HQL queries will converted to SQL queries by Dialect class for execution
- => If we change app from one DB to another DB then no need to change any query
- because Dialect class will take care of query conversion
- => HQL queries will give poor performance because of conversion (HQL -> SQL)

SQL : Structured Query Language (Database Dependent Queries)

- => In SQL, we will use table name & column names to write the query
- => SQL queries will directly execute in database

=> If we change app from one DB to another DB then all queries may not execute

=> SQL queries will give better performance than HQL

===== Repository with Custom Queries =====

```
public interface StudentRepository extends CrudRepository<Student, Integer> {
```

```
    @Query(value = "select * from student_dtls", nativeQuery = true)
```

```
    public List<Student> getAllStudents();
```

```
    @Query("from Student")
```

```
    public List<Student> getStudents();
```

```
}
```

=====

1) Using JPA pre-defined methods (Select + Non - Select)

2) Using findByXXX methods (Note: Only for select operations)

3) Using Custom Queries (Select + Non - Select)

=====Assignment=====

SQL : select * from student_dtls where student_gender=:gender

HQL : "from Student where gender = :gender"

SQL : select * from student_dtls where student_gender is null

HQL : "from Student where gender is null"

SQL : select * from student_dtls where student_rank >= : rank

HQL : from Student where rank >= :rank

SQL : select * from student_dtls where student_rank <= : rank

HQL : from Student where rank <= :rank

SQL : select * from student_dtls where student_gender=:gender and student_rank >= :rank

HQL : from Student where gender = :gender and rank >= :rank

SQL : select student_rank, student_gender from student_dtls

HQL : select rank, gender from Student

*****note:- (= :gender) in query after : do not enter a space

=====

Selection : Retrieving specific rows from the table. We can achieve this by using ' where ' keyword in the query

Ex : select * from student_dtls where gender = 'Male' ;

Projection : Retrieving specific columns from the table is called as Projection.

We can achieve by using column names in query.

Ex: select student_rank, student_gender from student_dtls

Note: We can combine selection & projection in single query.

Ex: select student_rank, student_gender from student_dtls where student_rank <= 100 ;

=====

JpaRepository

=====

=> It is predefined interface provided by Spring Data JPA

=> JpaRepository provided several methods to perform CRUD operations with database.

=> JpaRepository provided few additional methods to perform DB operations

JpaRepository = CrudRepository + PagingAndSorting + QueryByExample

=====

Pagination : Displaying table records in multiple pages is called as Pagination

=====

Ex:

=> Google Search Results will display with pagination (Page size : 10)

=> Gmail inbox mails will display with pagination (Page size : 50)

=> Flipkart products will display with pagination (Page size : 24)

=====Pagination Example=====

```
public class Application {  
    public static void main(String[] args) {  
        ConfigurableApplicationContext context = SpringApplication.run(Application.class,  
args);  
  
        StudentRepository studentRepo = context.getBean(StudentRepository.class);  
  
        Scanner s = new Scanner(System.in);  
  
        System.out.print("Enter Page Number");  
  
  
        int pageNo = s.nextInt();  
        int pageSize = 3;  
  
  
        // Page Num will start from 0  
        PageRequest pageReq = PageRequest.of(pageNo - 1, pageSize);  
        Page<Student> page = studentRepo.findAll(pageReq);  
        List<Student> students = page.getContent();  
        students.forEach(System.out::println);  
  
    }  
}
```

===== Sorting Example =====

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        ConfigurableApplicationContext context = SpringApplication.run(Application.class,
args);

        StudentRepository studentRepo = context.getBean(StudentRepository.class);
        List<Student> students = studentRepo.findAll(Sort.by("name").descending());
        students.forEach(System.out::println);
    }
}
```

=====

QueryByExample

=====

=> QBE is used to construct select query dynamically based on given entity object data

=> it used to filter a data based on our requirement

@SpringBootApplication

```
public class Application {
    public static void main(String[] args) {
        ConfigurableApplicationContext context = SpringApplication.run(Application.class,
args);

        StudentRepository studentRepo = context.getBean(StudentRepository.class);
        Student s = new Student();
        s.setGender("Male");
        s.setRank(90l);
        Example<Student> example = Example.of(s);
        List<Student> students = studentRepo.findAll(example);
        students.forEach(System.out::println);
    }
}
```

===== Assignment =====

=> Insert, Update & Delete records using Custom Queries in Data JPA

=> If we want to perform Non Select Operations using Data JPA custom query then we should use below 2 annotations at our method.

1) @Modifying

2) @Transactional

Note: The above annotations are not required for selection operation.

=====Example=====

```
public interface StudentRepository extends JpaRepository<Student, Integer> {
```

```
    @Query("delete from Student where id = :sid")
```

```
    @Modifying
```

```
    @Transactional
```

```
    public void deleteStudent(Integer sid);
```

```
    @Query("update Student set gender=:gender where id=:sid")
```

```
    @Modifying
```

```
    @Transactional
```

```
    public Integer updateStudent(Integer sid, String gender);
```

```
    @Query(value = "insert into student_dtls(student_id,student_name,student_gender) values(:id, :name, :gender)", nativeQuery = true)
```

```
    @Modifying
```

```
    @Transactional
```

```
    public void insertStudent(Integer id, String name, String gender);
```

```
    @Query("from Student")
```

```
    public void selectStudents();
```

```
}
```

=====

Timestamping in Data JPA

=====

=> @CreationTimestamp : It is used to populate record inserted date into DB column

=> @UpdateTimestamp : It is used to populate record updated date into DB column

@Entity

@Table(name = "STUDENT_DTLS") // optional

```
public class Student {  
    public Student() {  
    }  
    public Student(Integer id, String name, Long rank, String gender) {  
        this.id = id;  
        this.name = name;  
        this.rank = rank;  
        this.gender = gender;  
    }  
}
```

@Id

@Column(name = "student_id") // optional

private Integer id;

@Column(name = "student_name")

private String name;

@Column(name = "student_rank")

private Long rank;

@Column(name = "student_gender")

private String gender;

```

        @CreationTimestamp

        @Column(name = "CREATED_DATE", updatable = false)
        private LocalDateTime createDate;

        @UpdateTimestamp

        @Column(name = "UPDATED_DATE", insertable = false)
        private LocalDateTime updateDate;

        //setters & getters
    }

```

Note: LocalDate class represents will date value, where as LocalDateTime class will represent Date with Time.

=====

Soft Delete & Hard Delete

=====

=> Hard Delete means deleting the record from DB permanently using "Delete" query. Once we perform Hard Delete we can't get data back from DB.

=> Soft delete means updating the record as IN-ACTIVE. Soft Deleted records will present in DB so we can access whenever we want.

Ex : Active / De-Activate

Note: We can implement SOFT DELETE using additional column in DB table (ACTIVE_SW)

ACTIVE_SW =>>>> Y =====> Active record

ACTIVE_SW =>>>> N =====> Deleted record

===== **Soft Delete Example** =====

```

@SpringBootApplication
public class Application {

    public static void main(String[] args) {

        ConfigurableApplicationContext context = SpringApplication.run(Application.class,
args);

        StudentRepository studentRepo = context.getBean(StudentRepository.class);

```

```

        Student student = studentRepo.findById(101).get();

        student.setActiveSW("N");

        studentRepo.save(student);

    }

}

```

=====

Primary key & Composite Primary Key In DB

=====

=> Primary key is a constraint to maintain unique records in the table

Primary key = UNIQUE + NOT NULL

```

create table product (
    product_id integer not null,
    product_name varchar(255),
    product_price double precision not null,
    primary key (product_id)
);

```

=> If we try to insert duplicate value in PK column value then we will get UniqueConstraintException.

=> We shouldn't ask end users to enter value for PK column because users may enter duplicate value.

=> To generate value for PK column we will use GENERATOR concept.

===== MYSQL DB =====

AUTO / SEQUENCE / TABLE ==> New table will be created to maintain primary column values

IDENTITY ==> Will use AUTO_INCREMENT to generate value for Primary key column value

Note: MySQL DB will not support Sequences.

===== Oracle DB =====

AUTO => sequence to generate primary key value (default sequence name : hibernate_sequence)

SEQUENCE ==> We can configure our own sequence to generate PK value like below

```
create sequence pid_seq  
start with 1000  
increment by 1;
```

TABLE ==> New table will be created to maintain primary column values

Note: Oracle DB will not support for AUTO_INCREMENT

===== Configuring Table Generator =====

```
@Id  
@TableGenerator(initialValue = 100, name = "pid", table="pid_seq_tbl")  
@GeneratedValue(strategy = GenerationType.TABLE, generator="pid")  
private Integer productId;
```

===== Configuring Custom Sequence To Generate PK Value =====

=> First we need to create sequence in db like below

```
create sequence pid_seq  
start with 1000  
increment by 1;
```

=> Configure Custom Sequence in Entity class like below

```
@Id  
@SequenceGenerator(name = "pid", sequenceName = "pid_seq")  
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "pid")  
private Integer productId;
```

=====

Custom Generator : <https://youtu.be/lijGVtT9ZPk>

=====

=====

Composite Primary Key

=====

-> If a table contains more than one PK column then it is called Composite Primary key

create table product (

product_id integer not null,

product_name varchar(255),

product_price double precision not null,

primary key (product_id, product_name)

);

-> When we have Composite PKs then the combination PK columns data shouldn't be repeated in table.

Note: We can't use generators to generate value for Composite Primary keys.

===== **Entity Class** =====

@Embeddable

public class AccountPK implements Serializable{

private Integer accId;

private String accType;

private Long accNum;

// setters & getters

}

@Entity

public class Account {

private String holderName;

private String branch;

@EmbeddedId

private AccountPK accountPk;

//setters & getters}

=====Repository=====

```
public interface AccountRepository extends JpaRepository<Account, AccountPK> {
```

```
}
```

=====Boot start class=====

```
@SpringBootApplication
```

```
public class Application {
```

```
    public static void main(String[] args) {
```

```
        ConfigurableApplicationContext context = SpringApplication.run(Application.class,
args);
```

```
        AccountRepository accountRepo = context.getBean(AccountRepository.class);
```

```
        /*AccountPK pk = new AccountPK();
```

```
        pk.setAccId(2);
```

```
        pk.setAccType("Current");
```

```
        pk.setAccNum(32435668l);
```

```
        Account acc = new Account();
```

```
        acc.setHolderName("Raju");
```

```
        acc.setBranch("Ameerpet");
```

```
        acc.setAccountPk(pk);
```

```
        accountRepo.save(acc);*/
```

```
        AccountPK pk = new AccountPK();
```

```
        pk.setAccId(2);
```

```
        pk.setAccType("Current");
```

```
        pk.setAccNum(32435668l);
```

```

        Optional<Account> findById = accountRepo.findById(pk);

        System.out.println(findById.get());
    }
}

```

=====

Connection Pooling in Data JPA

=====

- => Connection Pooling is used to maintain connections required for our application.
- => Getting Connection from DB directly is not recommended because it will degrade our application.
- => We need to create connection pool to store connections when our application starts.
- => To perform DB operations we need to get connection from Connection Pool instead of getting from DB.
- => Using Connection Pool we can improve performance of the application.

Note: Spring Data JPA will use Hikari Connection Pool by default.

=====

Working with Stored Procedures

=====

- => Procedure means set of sql queries
- => Procedures are used to write business logic at database side
- => Procedures are used to improve performance of the application.

----- Procedure to retrieve data from Product table -----

```

DELIMITER $$

CREATE PROCEDURE getProducts()

BEGIN

    SELECT * FROM PRODUCT;

END$$

DELIMITER ;

call getProducts();

```

----- Calling Procedure using JpaRepository -----

```
public interface ProductRepository extends JpaRepository<Product, Integer>{

    @Query(value="call getProducts()", nativeQuery=true)
    public List<Product> getAllProducts();

}
```

- 1) What is Spring Data JPA ?
- 2) Hibernate Vs Data JPA ?
- 3) Spring Data JPA Repositories
- 4) What is Entity
- 5) Annotations to map Java class to DB table
- 6) How to develop application using Data JPA
- 7) CrudRepository methods
- 8) JpaRepository methods
- 9) CrudRepository vs JpaRepository
- 10) What is Pagination ?
- 11) Sorting
- 12) Query By Example
- 13) Working with findByXXX methods
- 14) Custom Queries using @Query (@Modifying, @Transactional)
- 15) HQL Vs SQL
- 16) What is Dialect ?
- 17) Calling Stored Procedures
- 18) Timestamping
- 19) Soft Delete Vs Hard Delete
- 20) Primary key
- 21) Generators

22) Composite Primary key (@Embeddeble, @EmbeddedId)

23) Connection Pooling (Hikari CP)

24) Spring Boot Profiles

25) How to develop Custom Generator

=====

Association Mapping / Relationships in DB tables

=====

=> We can divide DB side relations into 4 types

Note: To establish relationships between table we will use Foreign Key.

1) One To One (Ex: One Person will have one Passport)

2) One To Many (Ex : One Employee will have Multiple Addresses)

3) Many To One (Ex: Multiple Books belongs to one Author)

4) Many To Many (Ex: Multiple Users having Multiple Roles)

=> When DB tables are having Relation then we need to represent that relation in our Entity classes also.

=> The process of representing DB tables relation in Entity classes is called as Association Mapping.

Cascade Type : Default Type is NONE : It represents operations on parent record should reflect on child record or not

Ex: When we delete parent record then we want to delete all Child Records of that Parent.

Fetch Type : Default Type is LAZY : It represents whether load child records along with Parent or not

Ex: When we retrieve Parent record i want to retrieve all child records of that Parent.

=====OneToManyMapping=====

@Entity

@Data

public class Employee {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Integer empId;

 private String empName;

 private Double empSalary;

 @OneToMany(mappedBy = "emp", cascade = CascadeType.ALL, fetch = FetchType.EAGER)

 private List<Address> addr;

}

@Entity

@Data

public class Address {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Integer addrId;

 private String city;

 private String state;

 private String country;

 @ManyToOne

 @JoinColumn(name = "emp_id")

 private Employee emp;

}

public interface EmpRepository extends JpaRepository<Employee, Integer>{

}

```

public interface AddressRepository extends JpaRepository<Address, Integer>{

}

@SpringBootApplication
public class Application {

    public static void main(String[] args) {

        ConfigurableApplicationContext context = SpringApplication.run(Application.class,
args);

        EmpRepository empRepository = context.getBean(EmpRepository.class);
        AddressRepository addrReposiotry = context.getBean(AddressRepository.class);

        Employee e = new Employee();
        e.setEmpName("Raja");
        e.setEmpSalary(4000.00);

        Address a1 = new Address();
        a1.setCity("Hyd");
        a1.setState("TG");
        a1.setCountry("India");
        a1.setEmp(e);

        Address a2 = new Address();
        a2.setCity("GNT");
        a2.setState("AP");
        a2.setCountry("India");
        a2.setEmp(e);

        // setting addresses to emp
        List<Address> addrList = Arrays.asList(a1, a2);
        e.setAddr(addrList);
    }
}

```

```
// empRepository.save(e);
```

```
// empRepository.findById(2);
```

```
// empRepository.deleteById(1);
```

```
// addrReposiotry.findById(3);
```

```
}
```

```
}
```

=====

Spring Web MVC

=====

- > It is one module available in Spring Framework
- > Spring Web MVC is used to develop 2 types of applications

1) Web Applications (C 2 B)

Ex: Gmail, facebook, naukri

2) Distributed Applications (B 2 B)

Ex:

MakeMyTrip -----> IRCTC

Passport -----> AADHAR

- > Web & Distributed applications development made easy
- > Form Data Binding To Java Objects
- > Flexibility in Form Binding (Type casting will be done)
- > Form Validations (Server Validation)
- > Supports Multiple Presentation Technologies (Ex: Jsp & thymeleaf)
- > Embedded Servers (Default: Tomcat)

=====

Spring Web MVC Architecture

=====

- 1) Dispatcher Servlet : It acts as a front controller
- 2) HandlerMapper : It will identify which request should be processed by which controller and which method
- 3) Controller : It will handle request and decides response to send using ModelAndView object.
- 4) ModelAndView : Model represents data in key-value format. View Represents logical file name to display.
- 5) View Resolver : It is used to identify physical location of view files
- 6) View : It is used to render model data on view file.

=====

Building First Spring Web MVC Application

=====

1) Create Spring Boot application with below dependencies

- a) spring-boot-starter-web / spring web
- b) tomcat-embed-jasper
- c) spring-boot-devtools

2) Create Controller class using @Controller annotation

3) Write required methods in Controller class and bind them to Http Request Methods

4) Create Presentation file (jsp) with presentation logic

5) Configure View Resolver in application.properties file

6) Run the application and test it.

=====

=> tomcat-embed-jasper dependency :

=====

<dependency>

 <groupId>org.apache.tomcat.embed</groupId>

 <artifactId>tomcat-embed-jasper</artifactId>

</dependency>

=====

what is context path ?

=====

=> project name we can use as context path to differentiate the multiple project in the server

=> we can use properties file set context path as below

 -> server.servlet.context-path = / webapp

=====

client can send a data to server in tree ways

- 1) Query Parameters
- 2) Path Parameters
- 3) Request Body

=====

Query Parameters

=====

=> Query Parameters are use to send data from client to server URL

=> Query Parameters are called as Request parameters

=> Query Parameters will represents a data in key value format

=> Query Parameters should present only at end of the URL

=> Query Parameters will starts with '?' symbol

=> Query Parameeters will be sepreted by '&' symbol

Ex-1 : www.youtube.com/watch?v=jhp143u43y5u

www.ashokit.in?course=sbms&treiner=ashok

=> to read Query Parameters in spring controller we will use @RequestParam annotation

=====

Path Parameters

=====

=> Path Parametes are used to send data from client to server URL

=> Path Parameters are also called as URI Parameters

Note: Path Parameters will represent data directley (without key) like below

Ex: www.ashokit.in/course/{cname}/treiner/{tname}

=> Path Parametrs can present anywhere in the URL.

=> to read Path Parameters in Spring Controller we will use @PathVariable annotation

=====

Query Paramters / Request Params

=====

=> To send data to server in URL

=> Key Value format

=> will present At end of the URL

=> Starts with ? symbol

=> Will be seperated by & symbol

URL : www.ashokitech.com/course?name=sbms&trainer=ashok

=> To Read query params we will use @RequestParam annotation

=====Query Param Example=====

@Controller

```
public class BookController {  
    // http://localhost:8080/msg?name=ashok  
    @GetMapping("/msg")  
    public ModelAndView getMsg(@RequestParam String name) {  
        String msgTxt = name + ", Good Evening";  
        ModelAndView mav = new ModelAndView();  
        mav.addObject("msg", msgTxt);  
        mav.setViewName("index");  
        return mav;  
    }  
    // http://localhost:8080/book?name=spring&author=johnson  
  
    @GetMapping("/book")  
    public ModelAndView getBookData(@RequestParam String name, @RequestParam String  
author) {  
        System.out.println("Name :: " + name);  
        System.out.println("Author ::" + author);  
        ModelAndView mav = new ModelAndView();  
        mav.addObject("msg", name + " By " + author + " is out of stock...");  
        mav.setViewName("index");  
        return mav;  
    }  
}
```

=====

Path Paramters / URI Params

=====

- => To send data to server in URL
- => It will represent data directley (no keys)
- => Can present anywhere in uRL
- => Starts with / symbol

URL : www.ashokitech.com/course/sbms/

- => To Read query params we will use @PathVariable annotation

=====Path Param Example =====

@Controller

```
public class CarController {  
    // http://localhost:8080/car/101/hyd  
    @GetMapping("/car/{carId}/hyd")  
    public ModelAndView getCarColor(@PathVariable Integer carId) {  
        ModelAndView mav = new ModelAndView();  
  
        String color = null;  
  
        if (carId >= 100) {  
            color = "Red";  
        } else {  
            color = "Black";  
        }  
  
        mav.addObject("msg", "Car Color is :" + color);  
        mav.setViewName("index");  
        return mav;  
    }  
}
```

```
// http://localhost:8080/stock/benz/location/hyd
```

```
@GetMapping("/stock/{brand}/location/{loc}")
```

```
public ModelAndView getCarStock(@PathVariable String brand, @PathVariable String loc) {
```

```
    ModelAndView mav = new ModelAndView();
```

```
    mav.addObject("msg", "In " + loc + " " + brand + " cars Out Of Stock");
```

```
    mav.setViewName("index");
```

```
    return mav;
```

```
}
```

```
}
```

```
=====
```

We can develop controller methods in 2 ways

```
=====
```

1) By Taking ModelAndView as return type

2) By taking String as return type

```
=====2approches=====
```

```
@Controller
```

```
public class MyController {
```

```
    @GetMapping("/welcome")
```

```
    public ModelAndView getWelcomeMsg(@RequestParam String name) {
```

```
        String msgTxt = name + ", Welcome to Ashok IT..";
```

```
        ModelAndView mav = new ModelAndView();
```

```
        mav.addObject("msg", msgTxt);
```

```
        mav.setViewName("index");
```

```
        return mav;
```

```
}
```

```
    @GetMapping("/greet")
```

```
    public String getGreetMsg(@RequestParam String name, Model model) {
```

```
        model.addAttribute("msg", name+", Good Evening...!!");
```

```
        return "index";
```

```
}}
```

Note: Method return type is string which represents logical view name. Model is used to send data from controller to UI in key-value format.

=====

Forms development

=====

=> Forms are essential part in web applications

=> Forms are used to collect data from user to perform business operation

===== **index.jsp** =====

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
```

```
    pageEncoding="ISO-8859-1"%>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="ISO-8859-1">
```

```
<title>Insert title here</title>
```

```
</head>
```

```
<body>
```

```
    <font color='green'>${msg}</font>
```

```
    <h3>User Form</h3>
```

```
    <form action="user" method="POST">
```

```
        <table>
```

```
            <tr>
```

```
                <td>Name:</td>
```

```
                <td><input type="text" name="name" /></td>
```

```
            </tr>
```

```
            <tr>
```

```
                <td>Email:</td>
```

```
                <td><input type="email" name="email" /></td>
```

```
            </tr>
```

```

        <tr>
            <td>Phno:</td>
            <td><input type="number" name="phno" /></td>
        </tr>
        <tr>
            <td></td>
            <td><input type="submit" value="Submit" /></td>
        </tr>
    </table>
</form>
</body>
</html>

```

=====UserController.java=====

```

@Controller
public class UserController {
    @GetMapping("/")
    public String loadForm(HttpServletRequest req) {
        return "index";
    }
    @PostMapping("/user")
    public String handleSubmitBtn(User user, Model model) {
        System.out.println(user);
        // save in database
        model.addAttribute("msg", "User Saved");
        return "index";
    }
}

```

=====

=> Spring Web MVC module provided Form Tag library to simplify forms development

<form:form/>

<form:input/>

<form:password/>

<form:radiobutton/>

<form:select/>

<form:checkbox/>

=> To use web mvc form tag library we have to use below directive

<%@ taglib uri="http://www.springframework.org/tags/form" prefix="form"%>

=====

Requirement : Develop student web application with below functionality

1) Student Registration - Store in Database

- a) Name - textfield
- b) Email - textfield
- c) gender - radiobuttons
- d) course - drop down
- e) timings - checkboxes

2) View Students - display all registered students details in web page

=====

1) Create SpringBoot application with below dependencies

- a) web-starter
- b) datajpa-starter
- c) h2
- d) devtools
- e) jstl
- f) tomcat-embed-jasper

2) Create Entity class & repository interface to store the data

3) Create Controller class to handle request & response

4) Create View Files using JSP

5) Configure below properties in application.properties / yml file

6) Run the application and test it.

Project Code : https://github.com/ashokitschool/Spring_WEB_MVC_FORM_APP

=====

Thymeleaf

=====

=> Thymeleaf we can use as presentation technology in Spring Web MVC based applications.

=> Thymeleaf is an alternate of JSP

=> JSP files can't be executed in browser directly. JSP page should be converted into Servlet for execution.

=> Thymeleaf is a template engine which can be used in HTML pages directly

=> HTML pages will execute in browser directly.

HTML + Thymeleaf = Dynamic Web Pages

=> Performance wise Thymeleaf pages are faster than jsp pages

=> Thymeleaf introduced to overcome the problems of JSP.

=> To use Thymeleaf in boot application we need to add below starter

'spring-boot-starter-thymeleaf'

=====

Application Development with thymeleaf

=====

1) Create boot app with below dependencies

a) web-starter

b) thymeleaf-starter

c) devtools

2) Create Spring Controller with Required methods

3) Create Thymeleaf templates under src/main/resources/templates folder

(file extension is .html)

this is the thymeleaf xmlns (xml name space) :-> <html xmlns:th="http://www.thymeleaf.org">

4) Run the application and test it.

Git Repo URL : https://github.com/ashokitschool/springboot_thyemleaf_app.git

=====

How to configure Jetty as Embedded Server ?

=====

- 1) Exclude starter-tomcat from starter-web dependency
- 2) Add jetty-starter in pom.xml file

```
<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-web</artifactId>

    <exclusions>

        <exclusion>

            <groupId>org.springframework.boot</groupId>

            <artifactId>spring-boot-starter-tomcat</artifactId>

        </exclusion>

    </exclusions>

</dependency>

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-jetty</artifactId>

</dependency>
```

=====

How to send direct response from Spring Controller without using View Pages ?

=====

=> By using @ResponseBody annotation in Spring Controller class method we can send direct response to client.

@Controller

```
public class MessageController {

    @GetMapping("/welcome")

    @ResponseBody

    public String getWelcomeMsg() {
```

```

        return "Welcome to Ashok IT..!!";
    }

    @GetMapping("/greet")
    public String getGreetMsg(Model model) {
        model.addAttribute("msg", "Good Evening");
        return "index";
    }
}

```

Note : @Controller + @ResponseBody = @RestController

=> @RestController is used to send direct response to client without any view files.

=====

What is Interceptor in Spring Web MVC ?

=====

-> We can use Interceptor to perform pre-processing and post-processing of every request

Pre-Processing : Before Request Processing by Controller method

Post-Processing : After request processed by controller method

-> Using Interceptor we can trap each and every request

Use case for Interceptor

- 1) Calculate Each Request processing time
- 2) Log Each Request and Response details
- 3) Request Authentication etc...

=====

Exception Handling in Spring Web MVC

=====

- > Exception means un-expected and un-wanted situation
- > Exception distrubs normal flow of our application execution
- > When exception occurs then our program will terminate abnormally
- > As a developer we should handle exception to achieve graceful termination of our application.
- > To handle exceptions, Java provided below keywords

- 1) try
- 2) catch
- 3) throw
- 4) throws
- 5) finally

=> To handle Exceptions in Spring Web MVC application then we can create a method and we can use below annotation

`@ExceptionHandler`

=> When exception occurs then we will redirect user to error page like below.

`@ControllerAdvice`

```
public class GlobalExceptionHandler {  
  
    @ExceptionHandler(value = Exception.class)  
    public ModelAndView handleAE(Exception ex) {  
        ModelAndView mav = new ModelAndView();  
        mav.setViewName("page");  
        return mav;  
    }  
}
```

Note : here error represents our error page which display some message to client to try after sometime.

=====

Requirement:

=====

1) Develop one To-Do task application. Application should contains below functionalities

- a) User Registration (Name, Email, Pwd, Gender & Phno)
- b) User Login (Email & Pwd)
- c) Create Task (Task Name, Date, Timing)
- d) View Tasks

Note: Task Creation & Display Tasks functionality should work based on Logged in user.

e) Logout

=====

1) What is Spring Web MVC ?

2) Advantages of Spring Web MVC ?

3) Spring Web MVC Architecture

- DispatcherServlet
- HandlerMapper
- Controller
- ModelAndView
- ViewResolver
- View

4) What is Embedded Server (Ex : Tomcat & Jetty)

5) Building Web Application using Spring Boot

6) What is @Controller ?

7) What is @GetMapping & @PostMapping ?

8) What is Query Params and how to work with them ? (@RequestParam)

9) What is Path Params and how to work with them ? (@PathVariable)

10) Form Based Application Development

11) Spring Web MVC form tag library

12) What is Thymeleaf ?

- 13) Form Validation (validation-starter)
- 14) What is @ResponseBody annotation ?
- 15) How to make jetty as default embedded server.
- 16) How to configure H2 Database (Embedded Database)
- 17) Web app development using => Web MVC + Data JPA + H2 DB

=====

RESTful Services & Microservices

=====

Pre-Requisite : Spring Boot

=====

Course Content :

=====

Part- 1) RESTful Services

- => What is Distributed Application
- => Distributed Technologies
- => REST Introduction
- => REST Architecture
- => XML & JAX-B API.
- => JSON & Jackson API
- => HTTP Protocol (Methods + Status Codes)

- => REST Architecture Principle
- => REST API Development (Provider Development)
- => What is RestController
- => GET + POST + PUT + DELETE Methods
- => Query Params & Path Params
- => Request Body & Response Body
- => REST API Testing using POSTMAN
- => Swagger

- => Rest Client Development (Consumer Development)
- => RestTemplate class (Sync)
- => WebClient (Sync & Async)

- => Exception Handling in REST API

Part-2) Spring Security

- => Authentication
- => Authorization
- => Basic Authentication
- => OAuth 2.0
- => JWT

Part-3) Microservices with Spring Cloud

- => Monolith Architecture
- => Pros & Cons of Monolith
-
- => Microservices Introduction
- => Microservices Architecture
- => Pros & Cons of Microservices
-
- => Service Registry (Eureka Server)
- => Admin Server + Admin Client
- => Distributed Log Tracing (zipkin + sleuth)
- => API Gateway (Filters + Routing) - CloudGateway
-
- => Feign Client (Interservice Communication)
- => Load Balancer (Ribbon)
-
- => Circuit Breaker with Resilience
-
- => Config Server

Part-4 : Spring Boot - Integrations

- => Spring Boot with Redis Cache Integration

=> Spring Boot with Kafka Integration

=> Spring Boot with Docker Integration

=====

What is Distributed Application ?

=====

-> If one application is communicating with another application then they are called as Distributed Applications.

MakeMyTrip -----> IRCTC

Passport App -----> Aadhar App

Gpay / Phone Pay -----> Banking App

=> Distributed Applications are used for Business To Business Communication (B 2 B).

Note: Web Applications are used for Customer To Business Communication (C 2 B).

=====

Distributed Technologies

=====

1) CORBA

2) RMI

3) EJB

4) SOAP Based Webservices

5) RESTful Services (trending)

Provider : The application which is providing services to other application (Resource)

Consumer : The application which is accessing services from other application (Client)

=====

What is Interoperability ?

=====

=> Irrespective of language & platform if applications are communicating then they are called as Interoperable applications.

Java <-----> Python

Python <-----> .Net

.Net <-----> PHP

Note: By using RESTFUI Services we can develop Intereoperable applications.

=====

XML

=====

- > XML stands for Extensible Markup Language
- > XML Govenred by w3c org
- > XML is platfrom independent and language independent
- > XML is used to exchange data between applications (webservices)
- > XML represents data in elements format
- > Every Element contains open tag and closed tag

```
<id>101</id>
```

```
<name>Ashok</name>
```

- > In XML we can use 2 types of elements

1) Simple element

2) Compound element

- > The element which represents data directley is called as simple element.

```
<id>101</id>
```

- > The element which represents child element(s) is called as Compound Element.

```
<person>
```

```
<fname>Ashok</fname>
```

```
<lname>Kumar</lname>
```

```
</person>
```

=====

JAX-B API

=====

- => JAX-B stands for Java Architecture For XML binding
- => Using JAX-B API we can convert xml data to java object and java object to xml data.

Marshalling : Convert Java Object to XML data

Un-Marshalling : Convert XML data to Java Object

- => To perform Marshalling & Un-Marshalling we need to create Binding Classes.
- => Binding Class means the class which represents XML structure

Note: Upto JDK 1.8v JAX-B is part of JDK software. From JDK 1.9 version JAX-B removed from JDK.

Note: From java 1.9v or above versions should have jax-b dependency.

=====JAX-BExample=====

@XmlRootElement

```
public class Person {  
    private Integer id;  
    private String name;  
    private String email;  
    private String gender;  
    private Address addr;  
    //setters & getters  
}  
  
public class Address {  
    private String city;  
    private String state;  
    private String country;  
    //setters & getters  
}  
  
public class MarshallDemo {  
    public static void main(String[] args) throws Exception {  
        Address addr = new Address();  
        addr.setCity("Hyd");  
        addr.setState("TG");  
        addr.setCountry("India");  
  
        Person p = new Person();  
        p.setId(101);  
        p.setName("Ashok");  
        p.setGender("Male");  
        p.setEmail("ashokitschool@gmail.com");  
        p.setAddr(addr);  
    }  
}
```

```

        JAXBContext context = JAXBContext.newInstance(Person.class);

        Marshaller marshaller = context.createMarshaller();

        marshaller.marshal(p, new File("person.xml"));


        System.out.println("done.....");

    }
}

public class UnmarshalDemo {

    public static void main(String[] args) throws Exception {

        JAXBContext context = JAXBContext.newInstance(Person.class);

        Unmarshaller unmarshaller = context.createUnmarshaller();

        Person p = (Person) unmarshaller.unmarshal(new File("person.xml"));

        System.out.println(p);

    }
}

```

```

=====
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>
=====

```

=====

JSON

=====

=> JSON stands for Java Script Object Notation

=> JSON represents data in key-value format

=> JSON is light weight

=> JSON is Intereoperable (Platform & Language Independent)

=> We can use JSON structure to exchange data from one application to another application

Note: When compared with XML, JSON will take less memory

=> To work with JSON data we have below 3rd party APIs

1) Jackson (It is default in spring boot)

2) Gson (given by google)

=> Using above apis we can convert java object to json and json to java object

Serialization : Convert Java Object to JSON

De-Serialization : Convert JSON data to Java Object

=> Jackson API provided methods to perform operations java with json

ObjectMapper mapper = new ObjectMapper ();

mapper.writeValue(new File("person.json"), personObj);

Person p = mapper.readValue(new File("person.json"), Person.class);

===== Jackson API Example =====

```
public class Address {  
    private String city;  
    private String state;  
    private String country;  
    // setters & getters  
}
```

```
public class Passenger {  
    private String name;  
    private String from;  
    private String to;  
    private String gender;  
    private Address addr;  
    // setters & getters  
}  
  
public class JavaToJson {  
    public static void main(String[] args) throws Exception {  
  
        Address addr = new Address();  
        addr.setCity("Hyd");  
        addr.setState("TG");  
        addr.setCountry("India");  
  
        Passenger passenger = new Passenger();  
        passenger.setName("Raju");  
        passenger.setFrom("Hyd");  
        passenger.setTo("Delhi");  
        passenger.setGender("Male");  
        passenger.setAddr(addr);  
  
        ObjectMapper mapper = new ObjectMapper();  
        mapper.writeValue(new File("passenger.json"), passenger);  
  
        System.out.println("Done....");  
    }  
}
```

```

public class JsonToJava {

    public static void main(String[] args) throws Exception{

        ObjectMapper mapper = new ObjectMapper();

        Passenger passenger = mapper.readValue(new File("passenger.json"),
Passenger.class);

        System.out.println(passenger);

    }

}

```

=====GSONAPIExample=====

```

Gson gson = new Gson();

String json = gson.toJson(passenger);

System.out.println(json);

Passenger p = gson.fromJson(new FileReader("passenger.json"), Passenger.class);

System.out.println(p);

```

=====

HTTP Protocol

=====

- => HTTP stands for Hypertext Transfer Protocol
- => HTTP acts as mediator between Client & Server
- => HTTP is a stateless protocol. It will treat every request as a new request.

Note: To develop REST API we should know below details about HTTP Protocol.

- 1) HTTP Methods
- 2) HTTP Status Codes
- 3) Http Request
- 4) HTTP Response

=====

HTTP Methods

=====

GET ==> To get data from server to client

POST ==> To send data from client to server

PUT ==> To update data at server

DELETE ==> To delete data from server

Note: Every REST API method/endpoint should be binded to HTTP Protocol method

=> getTicketData () ---> HTTP GET Method ---> @GetMapping

=> bookTicket(..) ---> HTTP POST Method ----> @PostMapping

=> updateProduct (..) --> HTTP PUT Method ----> @PutMapping

=> deleteBook (..) ---> HTTP DELETE Method ---> @DeleteMapping

=====

HTTP Status Codes

=====

=> Server will send HTTP Status code to client in the response

=> HTTP Status codes will indicate how server processed our request

1XX (100 to 199) => Informational status code

2XX (200 to 299) => SUCCESS status code (OK)

3XX (300 to 399) => Redirectional

4XX (400 to 499) => Client Error

5XX (500 to 599) => Server Error

=====

HTTP Request Packet

=====

1) Request Line (HTTP Method + Request URL)

Ex: GET www.irctc.com/ticket/13454

2) Request Header (Meta data)

Ex :

Content-Type = application/json

Accept = application/json

Authentication = uname:pwd

Token = sldfjdsfyso

3) Request Body (Payload)

Ex: xml or json data

=====

HTTP Response

=====

1) Response Line (Https Status Code + Status Msg)

Ex: 200 OK

2) Response Header (Meta data)

Content-Type : application/json

Content-Length : 100

Date: mm/dd/yyyy

3) Response Body (Payload)

Ex: xml data or json data

=====

REST API Development

=====

=> It is very simple to develop REST API using Spring Boot

=> Spring Boot provided 'web-starter' to develop both web & distributed apps

=> 'Web-Starter' will provide tomcat as default embedded server

Step-1) Create Spring-Starter Project with below dependency

*** springboot-starter-web

Step-2) Create Rest Controller class using @RestController

Step-3) Write the required methods & bind them to HTTP Protocol Request

Step-4) Run the boot application (it will run in embedded server)

Step-5) Test our REST Application using POSTMAN tool

=====

Media Types

=====

consumes : It represents in which format REST API method can take input

produces : It represents in which format REST API method can provide output

Content-Type Header : It represents in which format client sending data to REST API in req body

Accept Header : It represents in which format client expecting response from REST API.

===== Media Types =====

@XmlRootElement

@Data

public class Customer {

private String name;

private String email;

private String gender;

}

@RestController

```
public class CustomerRestController {  
    @GetMapping(  
        value="/customer",  
        produces = {"application/xml" , "application/json"}  
    )  
    public Customer getCustomer() {  
        Customer c = new Customer();  
        c.setName("John");  
        c.setEmail("john@gmail.com");  
        c.setGender("Male");  
        return c;  
    }  
    @PostMapping(  
        value = "/customer",  
        consumes = {"application/xml", "application/json"},  
        produces = {"text/plain"}  
    )  
    public ResponseEntity<String> addCustomer(@RequestBody Customer customer) {  
        System.out.println(customer);  
        // logic to insert customer in db  
        return new ResponseEntity<>("Customer Saved", HttpStatus.CREATED);  
    }  
}
```

=====

Requirement : Develop REST API to book train tickets. It should contain 2 below endpoints

1) BOOK Ticket -- POST Request

Input : Passenger Data

Output : Ticket data

consumes : xml & json

produces : xml & json

2) Get Ticket -- GET Request

Input : Ticket ID

Output : Ticket Data

consumes : N/A

produces : xml & json

=====

Development Procedure For Above Requirement

=====

1) Create Spring Boot Application with below dependencies

a) web-starter

b) lombok

c) devtools

2) Create Request and Response Binding classes

3) Create REST Controller class with Required Methods

4) Bind Rest controller methods to HTTP Request methods

5) Run our application with Embedded Server

6) Test our application with POSTMAN.

=====

Passenger Data in JSON Format

=====

```
{  
  "fname": "ashok",  
  "lname": "kumar",  
  "from": "Hyd",  
  "to" : "Delhi",  
  "doj" : "30/03/2023",  
  "trainNum": "8878"  
}
```

=====

Swagger

=====

=> In Distributed applications two actors will be available

1) Provider

2) Consumer

=> Provider will be developed by one company

=> Consumer will be developed by another company

=> If consumer wants to access provider, consumer side dev team should know provider information

-> What is provider api url ?

-> What operations (methods) provider having ?

-> Operations are binded to which Request Type (GET or POST or PUT or DELETE)?

-> What input provider expecting from consumer ?

-> What output provider will give to consumer ?

-> Which data form provider will support for input and output ?

Note: If consumer side dev team having all the above information then only then can start consumer side development.

Note: Provider side dev team should provide API documentation to consumer side dev team.

=> Swagger is used to generate API documentation.

=> Swagger is a third party library which is used to generate REST API documentation.

Note: Using Swagger Documentation Consumer side dev team will understand Provider API information.

Steps to add Swagger Documentation for REST API

1) Add swagger & swagger-ui dependencies in project pom.xml

```
<dependency>

    <groupId>io.springfox</groupId>

    <artifactId>springfox-swagger2</artifactId>

    <version>2.6.1</version>

</dependency>

<dependency>

    <groupId>io.springfox</groupId>

    <artifactId>springfox-swagger-ui</artifactId>

    <version>2.6.1</version>

</dependency>
```

2) Create SwaggerConfig class to generate documentation

@Configuration

@EnableSwagger2

```
public class SwaggerConfig {

    @Bean

    public Docket apiDoc() {

        return new Docket(DocumentationType.SWAGGER_2)

            .select()

            .apis(RequestHandlerSelectors.basePackage("in.ashokit.rest"))

            .paths(PathSelectors.any())

            .build();

    }

}
```

3) Run the application and access Swagger documentation in browser.

Json Doc URL : <http://localhost:8080/v2/api-docs>

UI Doc URL : <http://localhost:8080/swagger-ui.html#/>

NOte: Using Swagger UI we can test REST API functionality just like POSTMAN.

- 1) What is Distributed Application
- 2) Distributed Technologies
- 3) What is Intereoperability ?
- 4) What is HTTP Protocol ?
- 5) HTTP Request Packet
- 6) HTTP Response Packet
- 7) HTTP Methods
- 8) HTTP Status Codes
- 9) XML & JAX-B API
- 10) JSON & JACKSON API / GSON api
- 11) REST API Development (@RestController)
- 12) REST API Testing with POSTMAN
- 13) MEDIA TYPES (conumes & produces)
- 14) Content-Type & Accept headers
- 15) Swagger Documentation
- 16) Query Parameter (@RequestParam)
- 17) Path Variable (@PathVariable)
- 18) @RequestBody
- 19) ResponseEntity (combine resp body + http status code)
- 20) REST API Deployment in AWS Cloud

=====

REST Client Development

{ up to v 52 video something }

=====

=> The application which is accessing other applications is called as REST Client.

=> In Spring Boot we can develop REST Client in 3 ways

- 1) RestTemplate class (Synchronus)
- 2) WebClient interface (Synchronus & Async) (introduced in spring 5.x version)
- 3) FeiginClient interface (spring cloud)

Note: From Spring 5.x version onwards we have to use WebClient instead of RestTemplate.

-----application.properties-----

spring.mvc.view.prefix=/views/

spring.mvc.view.suffix=.jsp

irctc.endpoint.book.ticket=http://3.110.190.17:8080/ticket

irctc.endpoint.get.ticket=http://3.110.190.17:8080/ticket/{ticketId}

@Service

public class MakeMyTripService {

 @Value("\${irctc.endpoint.book.ticket}")

 private String IRCTC_BOOK_TICKET_URL;

 @Value("\${irctc.endpoint.get.ticket}")

 private String IRCTC_GET_TICKET_URL;

 public Ticket getTicketInfo(String ticketId) {

 RestTemplate rt = new RestTemplate();

 ResponseEntity<Ticket> responseEntity =

 rt.getForEntity(IRCTC_GET_TICKET_URL, Ticket.class, ticketId);

 int status = responseEntity.getStatusCodeValue();

 if (status == 200) {

 Ticket ticket = responseEntity.getBody();

 return ticket;

 }

 return null;

 }

```

    public Ticket processTicketBooking(Passenger passenger) {
        RestTemplate rt = new RestTemplate();
        ResponseEntity<Ticket> responseEntity =
            rt.postForEntity(IRCTC_BOOK_TICKET_URL, passenger,
Ticket.class);
        int statusCode = responseEntity.getStatusCodeValue();
        if (statusCode == 200) {
            Ticket ticket = responseEntity.getBody();
            return ticket;
        }
        return null;
    }
}

```

IRCTC API Doc URL : <http://3.110.190.17:8080/swagger-ui.html>

=====

WebClient

=====

- > WebClient is a predefined interface introduced in Spring 5.x version
- > Using WebClient interface we can develop REST Client logics
- > WebClient supports both sync & async communication.

Sync : Blocking Thread (After sending request we have to wait for response)

Async : Non Blocking Thread (After sending request we no need to wait for response)

RestTemplate (C) : spring-boot-starter-web

WebClient (I) : spring-boot-starter-webflux

=====RestClientdevelopment with WebClient =====

@Service

public class MakeMyTripService {

 @Value("\${irctc.endpoint.book.ticket}")

 private String IRCTC_BOOK_TICKET_URL;

 @Value("\${irctc.endpoint.get.ticket}")

 private String IRCTC_GET_TICKET_URL;

 public Ticket getTicketInfo(String ticketId) {

 WebClient webClient = WebClient.create(); // get WebClient instance

 Ticket ticket = webClient.get() // represents HTTP GET request

 .uri(IRCTC_GET_TICKET_URL, ticketId) // ENDPOINT URL

 .accept(MediaType.APPLICATION_JSON)

 .retrieve() // take resp from response body

 .bodyToMono(Ticket.class) // bind resp body data to java obj

 .block(); // make sync call

 if(ticket!=null) {

 return ticket;

 }

 return null;

 }

 public Ticket processTicketBooking(Passenger passenger) {

 WebClient webClient = WebClient.create(); // get WebClient instance

 Ticket ticket = webClient.post()

 .uri(IRCTC_BOOK_TICKET_URL)

 .body(BodyInserters.fromValue(passenger))

 .header("Content-Type","application/json")

```

        .accept(MediaType.APPLICATION_JSON)
        .retrieve()
        .bodyToMono(Ticket.class)
        .block();

    if(ticket!=null) {
        return ticket;
    }
    return null;
}
}

```

=====

Async Client Development

=====

```

package in.ashokit;

@Service

public class MakeMyTripService {

    @Value("${irctc.endpoint.book.ticket}")
    private String IRCTC_BOOK_TICKET_URL;

    @Value("${irctc.endpoint.get.ticket}")
    private String IRCTC_GET_TICKET_URL;

    public void getTicketInfoSync(String ticketId) {

        System.out.println("Sync - method started....");

        WebClient client = WebClient.create();

        String response = client.get()

            .uri(IRCTC_GET_TICKET_URL, ticketId)

            .accept(MediaType.APPLICATION_JSON)

            .retrieve()

            .bodyToMono(String.class)

            .block(); // wait for response
    }
}

```

```
        System.out.println(response);

        System.out.println("Sync - method ended....");
    }

    public void getTicketAsync(String ticketId) {

        System.out.println("Async method execution started.....");

        WebClient client = WebClient.create();

        client.get()
            .uri(IRCTC_GET_TICKET_URL, ticketId)
            .accept(MediaType.APPLICATION_JSON)
            .retrieve()
            .bodyToMono(String.class)
            .subscribe(response -> handleResponse(response));

        System.out.println("Async method execution ended.....");
    }

    public void handleResponse(String response) {
        System.out.println(response);
    }
}
```

=====

=====

Spring Data REST

{ UpTo 56-video}

=====

=> It is used to simplify REST API development

=> We no need to create REST Controllers to perform CRUD operations with DB table when we use Spring Data REST.

=> To use Data-REST in our project we need to add below dependency in pom.xml (REST Repositories)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
```

@Entity

@Table(name = "book_tbl")

@Data

public class Book {

@Id

private Integer id;

private String name;

private Double price;

}

@RepositoryRestResource(path = "books")

public interface BookRepository extends JpaRepository<Book, Integer> {

public List<Book> findByNameContaining(@Param("name") String name);

}

@Configuration

```
public class MyDataRestConfig implements RepositoryRestConfigurer {
```

```
    @Override
```

```
        public void configureRepositoryRestConfiguration(RepositoryRestConfiguration config,
        CorsRegistry cors) {
```

```
            HttpMethod[] unsupportedMethods = { HttpMethod.PUT, HttpMethod.DELETE };
```

```
            config.getExposureConfiguration()
```

```
                .forDomainType(Book.class)
```

```
                .withItemExposure((metadata, http) -> http.disable(unsupportedMethods))
```

```
                .withCollectionExposure((metadata, http) -> http.disable(unsupportedMethods));
```

```
        }
```

```
    }
```

=====

REST API Exception Handling

=====

-> Exception means un-expected and un-wanted situation

-> Exception will cause abnormal termination of our program

-> To achieve graceful termination, we need to handle exceptions in our application

-> In springboot, we can handle exceptions in 2 ways

1) Local Exception Handling

2) Global Exception Handling

=====

Steps to implement Exception Handling

=====

- 1) Create boot application with web starter
 - 2) Create RestController with required method
 - 3) Create User Defined Exception class
 - 4) Create ExceptionInfo binding class
 - 5) Create Rest Controller Advice to handle global exceptions in our application.
-

```
package in.ashokit.exception;
```

```
public class CustomerNotFoundException extends RuntimeException {
```

```
    public CustomerNotFoundException() {  
        // TODO Auto-generated constructor stub  
    }
```

```
    public CustomerNotFoundException(String msg) {  
        super(msg);  
    }
```

```
}
```

```
@Data
```

```
public class ExceptionInfo {
```

```
    private String code;  
    private String msg;  
    private LocalDateTime date;
```

```
}
```

@RestControllerAdvice

public class AppExceptionHandler {

 @ExceptionHandler(value = CustomerNotFoundException.class)

 public ResponseEntity<ExceptionInfo> handleCnfe(CustomerNotFoundException cnfe) {

 ExceptionInfo info = new ExceptionInfo();

 info.setCode("EX0011");

 info.setMsg(cnfe.getMessage());

 info.setDate(LocalDateTime.now());

 return new ResponseEntity<>(info, HttpStatus.BAD_REQUEST);

 }

}

@Service

public class CustomerService {

 public String getCustomerNameById(Integer customerId) {

 if (customerId >= 100) {

 return "John";

 } else {

 throw new CustomerNotFoundException("Invalid customer id");

 }

 }

}

@RestController

public class CustomerRestController {

 @Autowired

 private CustomerService service;

 @GetMapping("/customer/{customerId}")

 public String getCustomerName(@PathVariable Integer customerId) throws Exception {

 return service.getCustomerNameById(customerId);

 }

}

REST Architecture Principles

REST : Representation State Transfer

- 1) Client Server Architecture (B 2 B)
- 2) No State / Session Management
- 3) Unique Addressability
- 4) Map REST API endpoints to HTTP Methods
- 5) MediaType Representation (consumes, produces & Content-Type, Accept)
- 6) HATEOS (Hypermedia as the Engine of Application State)

HATEOS

-> It is one of the REST Architecture Principle

-> It is used to send response along with hyperlinks for related data

Ex:

URL : <http://localhost:8080/customer/101>

```
{
    "id" : 101,
    "name" : "John",
    "email" : "john@gmail.com",
    "links" :{
        "url" : "http://localhost:8080/customers"
    }
}
```

=> In springboot we have HATEOS starter to develop REST API with HATEOS concept.

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>
```

@Data

@AllArgsConstructor

@NoArgsConstructor

```
public class User extends RepresentationModel<User> {
```

```
    private Integer id;
    private String name;
    private String email;
```

```
}
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import static org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;
```

```
import in.ashokit.binding.User;

@RestController

public class UserRestController {

    @GetMapping("/user")

    public ResponseEntity<User> getUser() {

        User user = new User(101, "John", "john@gmail.com");

        user.add(linkTo(methodOn(UserRestController.class).getUser()).withSelfRel());

        return new ResponseEntity<>(user, HttpStatus.OK);

    }

}
```

PUT -> To update complete resource data (user : name, age, dob)

PATCH -> To update particular fields in resource (order : status = Delivered)

=====

Microservices

=====

+++++

What is Monolith Architecture

+++++

-> If we develop all the functionalities in single project then it is called as Monolith architecture based application

-> We will package our application as a jar/war to deploy into server

-> As monolith application contains all functionalities, it will become fat jar/war

+++++

Advantages

+++++

- 1) Simple to develop
- 2) Everything is available at once place
- 3) Configuration required only once

+++++

Dis-Advantages

+++++

- 1) Difficult to maintain
- 2) Dependencies among the functionalites
- 3) Single Point Of Failure
- 4) Entire Project Deployment

***** To overcome the problems of Monolith, Microservices architecture came into market*****

-> Microservices is not a programming language

-> Microservices is not a framework

-> Microservices is not an Specification API

-> Microservices is an architectural design pattern

-> Microservices suggesting to develop application functionalities with loosely coupling

-> In Microservices architecture we don't develop all the functionalities in single project. We will divide project functionalities into several REST APIs

*******Note: One REST API is called as one Microservice*******

-> Microservices architecture based project means collection of REST APIs.

-> Microservices is not related to only java. Any programming language specific project can use Microservices Architecture.

+++++

Advantages

+++++

- 1) Loosely Coupling
- 2) Easy To maintain
- 3) Faster Development
- 4) Quick Deployment
- 5) Faster Releases
- 6) Less Downtime
- 7) Technology Independence

+++++

Dis-Advantages

+++++

- 1) Bounded Context
- 2) Lot of configurations
- 3) Visibility
- 4) Pack of cards

+++++

Microservices Architecture

+++++

-> We don't have any fixed architecture for Microservices

-> People are customizing microservices architecture according to their requirement

-> Most of the projects will use below components in Microservices Architecture

- 1) Service Registry (Eureka Server)
- 2) Services (REST APIs)
- 3) Interservice Communication (FeginClient)
- 4) API Gateway (Zuul Proxy)
- 5) Admin Server
- 6) Sleuth & Zipkin Server

+++++

Service Registry

+++++

- > Service Registry acts as DB of services available in the project
- > It provides the details of all the services which are registered with Service Registry
- > We can identify how many services available in the project
- > We can identify how many instances available for each service
- > We can use "Eureka Server" as service registry
- > Eureka Server provided by "Spring Cloud Netflix" library

+++++

Services

+++++

- > Services means REST APIs / Microservices
- > Services contains backend business logic
- > In the project, some services will interact with DB
- > In the project, some services will interact with third party REST API (external communication)
- > In the project, some services will interact with another services with in the project
(inter-service communication)
- > For inter-service communication we will use feign-client
- > To distribute the load, we can run one service with Multiple Instances (Load Balancing)

Note: We will register every service with Service Registry

+++++

API Gateway

+++++

- > API Gateway is used to manage our backend apis of the project
- > API Gateway acts as mediator between end users and backend apis
- > API Gateway can filter logic to decide request processing
- > API Gateway will contain Routing logic (which request should go to which REST API)
- > API Gateway also will be registered with Service Registry

+++++

Mini Project Implementation using Microservices Architecture

+++++

- 1) Service Registry (Eureka Server)
- 2) Spring Boot Admin Server (To monitor & manage boot applications)
- 3) Zipkin Server (Distributed Log Tracing) (<https://zipkin.io/pages/quickstart.html>)

+++++

Steps to develop Service Registry Application (Eureka Server)

+++++

- 1) Create Service Registry application with below dependency
 - a) EurekaServer (spring-cloud-starter-netflix-eureka-server)
 - b) web-starter
 - c) devtools

- 2) Configure @EnableEurekaServer annotation in boot start class

- 3) Configure below properties in application.yml file

server:

port: 8761

eureka:

client:

register-with-eureka: false

Note: If Service-Registry project port is 8761 then clients can discover service-registry and will register automatically with service-registry. If service-registry project running on any other port number then we have to register clients with service-registry manually.

4) Once application started we can access Eureka Dashboard using below URL

URL : <http://localhost:8761/>

+++++

Steps to develop Spring Boot Admin Server Project

+++++

1) Create Boot application with below dependencies

- a) web-starter
- b) devtools
- c) admin-server (codecentric)

2) Configure @EnableAdminServer annotation at boot start class

3) Configure the port number and run the application (port : 1111)

4) After application started, access Admin Server UI using app-url

URL : <http://localhost:1111/>

+++++

Steps to work with Zipkin Server

+++++

1) Download Zipkin server jar from website

URL : <https://zipkin.io/pages/quickstart.html>

2) Run the zipkin server jar from command prompt

Cmd : java -jar <jar-file-name>

Note: Zipkin server will run on 9411 port number

3) Access Zipkin server dashboard in browser

URL : <http://localhost:9411/>

+++++

Steps to develop GREET-API

+++++

1) Create Spring Boot application with below dependencies

- eureka-discovery-client
- starter-web
- devtools
- actuator
- sleuth
- zipkin
- admin-client

2) Configure @EnableDiscoveryClient annotation at start class

3) Create RestController with required method

4) Configure below properties in application.yml file

-----application.yml-----

server:

port: 9090

spring:

application:

name: GREET-API

boot:

admin:

client:

url: http://localhost:8080/

eureka:

client:

serviceUrl:

defaultZone: http://localhost:8761/eureka

management:

endpoints:

web:

exposure:

include: '*'

5) Run the application and check in Eureka Dashboard (It should display in eureka dashboard)

6) Check Admin Server Dashboard (It should display) (we can access application details from here)

Ex: Beans, loggers, heap dump, thread dump, metrics, mappings etc...

7) Send Request to REST API method

8) Check Zipkin Server UI and click on Run Query button

(it will display trace-id with details)

+++++

Steps To Develop WELCOME-API

+++++

1) Create Spring Boot application with below dependencies

- web-starter
- devtools
- eureka-discovery-client
- feign-client
- admin-client
- zipkin-client
- sleuth
- actuator

2) Configure @EnableDiscoveryClient & @EnableFeignClients annotations at boot start class

3) Create FeignClient to access GREET-API

```
@FeignClient(name = "GREET-API")
```

```
public interface GreetApiClient {
```

```
    @GetMapping("/greet")
```

```
    public String invokeGreetApi();
```

```
}
```

4) Create RestController with required method

Note: In Rest Controller we should have logic to access another REST API (GREET-API)

-> For Interservice Communication we will use FeignClient

-> Using FeginClient we can make rest call to another service using name of the service (no need of url)

-> FeginClient will get service URL from service-registry based on service-name

@RestController

```
public class WelcomeRestController {  
    private Logger logger = LoggerFactory.getLogger(WelcomeRestController.class);  
  
    @Autowired  
    private GreetApiClient greetClient;  
  
    @GetMapping("/welcome")  
    public String welcomeMsg() {  
  
        logger.info("welcomeMsg() execution - start");  
  
        String welcomeMsg = "Welcome to Ashok IT..!!";  
  
        String greetMsg = greetClient.invokeGreetApi();  
  
        logger.info("welcomeMsg() execution - end ");  
  
        return greetMsg + ", " + welcomeMsg;  
    }  
}
```

5) Configure below properties in application.yml file

```
server:
  port: 9091
spring:
  application:
    name: WELCOME-API
  boot:
    admin:
      client:
        url: http://localhost:1111/
  management:
    endpoints:
      web:
        exposure:
          include: '*'
```

6) Run WELCOME-API project (it should register in Eureka and Admin server)

7) Send Request to welcome-api (it should final response)

8) Verify Zipkin Server Dashboard for log tracing

+++++

-> We are running Service Registry project with Eureka Server on 8761 port number

-> Eureka Discovery Client applications are auto-registering with Eureka Server when port is 8761

-> If we change Eureka Server port number then we have to register Eureka Client application with Eureka Server using below property in application.yml file

```
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:9090/eureka
```

Note: We should configure this property in eureka client application.yml file

+++++

GREET API URL : DESKTOP-BDG00U7:GREET-API:9090/

WELCOME API URL : DESKTOP-BDG00U7:WELCOME-API:9091/

=====

API Gateway

=====

-> API Gateway will act as mediator between client requests & backend apis

-> API Gateway will provide single endpoint to access our backend apis

-> In API Gateway we will write mainly below 2 types of logics

1) Filters

2) Routing

-> Filters are used to execute some logic before request processing and after request processing

-> Routing is used to tell which request should go to which REST API

-> In Spring Cloud, we have 2 options to create API Gateway

1) Zuul Proxy (old approach)

2) Spring Cloud Gateway (latest approach)

Note: Zuul Proxy is not supported by latest versions of Spring Boot

+++++

Working with Spring Cloud API Gateway

+++++

1) Create Spring Boot application with below dependencies

-> web-starter

-> eureka-client

-> cloud-gateway

-> devtools

2) Configure @EnableDiscoveryClient annotation at boot start class

3) Configure API Gateway Routings in application.yml file like below

-----application.yml file-----

```
spring:
  cloud:
    gateway:
      discovery.locator:
        enabled: true
        lowerCaseServiceId: true
      routes:
        - id: welcome-api
          uri: lb://WELCOME-API
          predicates:
            - Path=/welcome
        - id: greet-api
          uri: lb://GREET-API
          predicates:
            - Path=/greet
      application:
        name: CLOUD-API-GATEWAY
  server:
    port: 3333
```

In API gateway we will have 3 types of logics

- 1) Route
- 2) Predicate
- 3) Filters

-> Routing is used to defined which request should be processed by which REST API in backend. Routes will be configured using Predicate

-> Predicate : This is a Java 8 Function Predicate. The input type is a Spring Framework ServerWebExchange. This lets you match on anything from the HTTP request, such as headers or parameters.

-> Filters are used to manipulate incoming request and outgoing response of our application

Note: Using Filters we can implement security also for our application.

@Component

```
public class MyPreFilter implements GlobalFilter {

    private Logger logger = LoggerFactory.getLogger(MyPreFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {

        logger.info("MyPreFilter :: filter () method executed...");

        // Accessing HTTP Request information
        ServerHttpRequest request = exchange.getRequest();

        HttpHeaders headers = request.getHeaders();
        Set<String> keySet = headers.keySet();

        keySet.forEach(key -> {
            List<String> values = headers.get(key);
            System.out.println(key + " :: "+values);
        });

        return chain.filter(exchange);
    }
}
```

+++++

- > We can validate client given token in the request using Filter for security purpose
- > We can write request and response tracking logic in Filter
- > Filters are used to manipulate request & response of our application
- > Any cross-cutting logics like security, logging, monitoring can be implemented using Filters

=====

Sleuth & Zipkin

=====

- > Microservices application means several REST APIs will be available
- > As part of application execution one Rest API can communicate with another REST API
- > When we send request from UI, it will process by Multiple REST APIs with Interservice communication

*** How we can understand which rest api is taking more time to process our request ? ***

- > If we add Sleuth dependency in REST API then it will add span-id and trace-id for log messages
- > For every request once trace-id will be generated by Sleuth
- > If one request is processing multiple REST API then Sleuth will use same span-id for REST APIs to generate log message
- > Trace-id is specific to one REST API
- > By using span-id and trace-id we can understand which REST api has taken more time process request
- > To monitor span-id and trace-id details we will use ZipKin server
- > Zipkin server is providing user interface (UI) to monitor all the details

Note: The REST APIs which are having sleuth dependency should register with Zipkin server

Note: By using Sleuth and Zipkin we achieve Distributed Log Tracing

=====

Steps to work with Sleuth and Zipkin

=====

- 1) Add below dependency in welcome-api and greet-api projects pom.xml

```
<dependency>  
    <groupId>org.springframework.cloud</groupId>  
    <artifactId>spring-cloud-starter-sleuth</artifactId>
```

</dependency>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-sleuth-zipkin</artifactId>

</dependency>

2) Download zipkin-server jar file (<https://zipkin.io/pages/quickstart>)

3) Run zipkin-server using "java -jar <zipkin-jar-filename>"

Note: Zipkin server runs on 9411 port

4) Run spring boot applications and send a request to rest controller method

5) Verify boot application logs display in console (span-id and trace-id will be attached to logs)

6) Go to Zipkin server dashboard and monitor event details

(URL : <http://localhost:9411>)

+++++

1) What is Monolith Architecture ?

2) Pros and Cons of Monolith Architecture

3) Microservices Introduction

4) Pros and Cons of Microservices

5) Microservices Architecture

6) Service Registry (Eureka)

7) Admin Server (Monitor & Manager actuators)

8) Zipkin Server with Sleuth

9) Backend Apis Development

10) Inter-service communication (Feign Client)

11) Load Balancing with Ribbon

12) Api Gateway (Front end gate of all backend apis)

13) Filters & Routings in API Gateway

=====

Cloud Config Server

=====

=> As of now we are configuring properties in application.properties or application.yml file

Ex: DB Props, SMTP props, Kafka Props, Messages etc...

=> application.properties or application.yml file will be packaged along with our application.

=> If we want to make any changes to properties then we have to re-package our application

=> To externalize properties from the application we can use Cloud Config Server

=====

Config Server App

=====

1) Create Git Repository and keep ymls files required for projects

Note: We should keep file name as application

app name : greet then file name : greet.yml

app name : welcome then file name : welcome.yml

Git Repo : https://github.com/ashokitschool/configuration_properties

2) Create Spring Starter application with below dependency

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-config-server</artifactId>

</dependency>

3) Write @EnableConfigServer annotation at boot start class

@SpringBootApplication

@EnableConfigServer

public class Application {

public static void main(String[] args) {

SpringApplication.run(Application.class, args); } }

4) Configure below properties in application.yml file

spring:

cloud:

config:

server:

git:

uri: https://github.com/ashokitschool/configuration_properties

clone-on-start: true

management:

security:

enabled: false

5) Run Config Server application

=====

Config Server Client Development

=====

1) Create Spring Boot application with below dependencies

a) web-starter

b) config-client

c) dev-tools

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-starter-config</artifactId>

</dependency>

2) Create Rest Controller with Required methods

@RestController

@RefreshScope

public class WelcomeRestController {

 @Value("\${msg}")

 private String msg;

 @GetMapping("/")

 public String getWelcomeMsg() {

 return msg;

 }

}

3) Configure ConfigServer url in application.yml file like below

server:

 port: 9090

spring:

 config:

 import: optional:configserver:http://localhost:8080

 application:

 name: greet

4) Run the application and test it.

=====

Circuit Breaker

- > Circuit Breaker is a design pattern in Microservices
- > Circuit Breaker is used to implement fault-tolerance systems
- > Fault-tolerance systems are also called as resilience systems
- > Fault-tolerance system means when main logic is failed to execute then we should execute fallback logic to process client request

Usecase

=> Get data from redis, if redis logic is failing then we should get data from database

Note: If redis logic is failing for 3 requests continuously then execute db logic for 30 mins. After 30 mins re-try for redis logic execution if it is working then execute redis logic only. If 3 re-try executions failed with redis then execute db logic for next 30 mins.

- > To implement circuit-breaker we should add below dependency in pom.xml file

```
<dependency>
    <groupId>io.pivotal.spring.cloud</groupId>
    <artifactId>spring-cloud-services-starter-circuit-breaker</artifactId>
</dependency>
```

- > Write @EnableHystrix annotation at boot start class

@RestController

```
public class DataRestController {
```

```
    @GetMapping("/data")
```

```
    @HystrixCommand(
```

```
        fallbackMethod = "getDataFromDB",
```

```

        commandProperties = {

            @HystrixProperty(name="circuitBreaker.requestVolumeThreshold", value="3"),

            @HystrixProperty(name="circuitBreaker.sleepWindowInMilliseconds", value="10000"),

            @HystrixProperty(name="circuitBreaker.enabled",
value="true")

        }

    )

    public String getDataFromRedis() {

        System.out.println("**getDataFromRedis() method called**");

        if (new Random().nextInt(10) <= 10) {

            throw new RuntimeException("Redis Server Is Down");

        }

        // logic to access data from redis

        return "data accessed from redis (main logic) ....";

    }

    public String getDataFromDB() {

        System.out.println("**getDataFromDB() method called**");

        // logic to access data from db

        return "data accessed from database (fall back logic) ....";

    }

}

```

=====

Spring Security

How to secure REST APIs using Spring Boot

- > Security is very important for every web application
- > To protect our application & application data we need to implement security logic
- > Spring Security concept we can use to secure our web applications / REST APIs
- > To secure our spring boot application we need to add below starter in pom.xml file

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

Note: When we add this dependency in pom.xml file then by default our application will be secured with basic authentication. It will generate random password to access our application.

Note: Generated Random Password will be printed on console.

- > We need to use below credentials to access our application

Username : user

Password : <copy the pwd from console>

- > When we access our application url in browser then it will display "Login Form" to authenticate our request.

- > To access secured REST API from postman, we need to set Auth values in POSTMAN to send the request

Auth : Basic Auth

Username : user

Password : <copy-from-console>

How to override Spring Security Default Credentials

-> To override Default credentials we can configure security credentials in application.properties file or application.yml file like below

```
spring.security.user.name=ashokit
```

```
spring.security.user.password=ashokit@123
```

-> After configuring credentials like above, we need to give above credentials to access our application / api.

How to secure specific URL Patterns

-> When we add 'security-starter' in pom.xml then it will apply security filter for all the HTTP methods of our application.

-> But in reality we need to secure only few methods not all methods in our application.

For Example

/ login-page --> security not required

/ transfer ---> security required

/ balance ---> security required

/about-us ---> security not required

-> In order to achieve above requirement we need to Customize Security Configuration in our project like below

@Configuration

@EnableWebSecurity

public class SecurityConfigurer {

 @Bean

 public SecurityFilterChain securityFilter(HttpSecurity http) throws Exception{

 http.authorizeHttpRequests((request) -> request

 .antMatchers("/", "/login", "/about", "/swagger-ui.html").permitAll()

 .anyRequest().authenticated()

).formLogin();

 return http.build();

 }

}

=====

Spring Boot Security with JDBC Authentication

=====

Step-1) Setup Database tables with required data

-- users table structure

CREATE TABLE `users` (

 `username` VARCHAR(50) NOT NULL,

 `password` VARCHAR(120) NOT NULL,

 `enabled` TINYINT(1) NOT NULL,

 PRIMARY KEY (`username`)

);

-- authorities table structure

```
CREATE TABLE `authorities` (
  `username` VARCHAR(50) NOT NULL,
  `authority` VARCHAR(50) NOT NULL,
  KEY `username` (`username`),
  CONSTRAINT `authorities_ibfk_1` FOREIGN KEY (`username`)
    REFERENCES `users` (`username`)
);
```

===== Online Encrypt : <https://bcrypt-generator.com/> =====

-- insert records into table

```
insert into users values ('admin',
'$2a$12$e9olZjBeSJDryJ/P5p1Ep.WPzJ3f4.C2vHC/as1E22R25XXGpPYyG', 1);
insert into users values ('user',
'$2a$12$JQIGAJhdSOoTXAzIpbDxpemXcYHCmxYOnodLNBeNORH8J4FLxHGvK', 1);
```

```
insert into authorities values ('admin', 'ROLE_ADMIN');
```

```
insert into authorities values ('admin', 'ROLE_USER');
```

```
insert into authorities values ('user', 'ROLE_USER');
```

Step-2) Create Boot application with below dependencies

- a) web-starter
- b) security-starter
- c) data-jdbc
- d) mysql-connector
- e) lombok
- f) devtools

Step-3) Configure Data source properties in application.yml file

spring:

datasource:

driver-class-name: com.mysql.cj.jdbc.Driver

password: AshokIT@123

url: jdbc:mysql://localhost:3306/sbms27

username: ashokit

jpa:

show-sql: true

Step-4) Create Rest Controller with Required methods

@RestController

public class UserRestController {

 @GetMapping(value = "/admin")

 public String admin() {

 return "<h3>Welcome Admin :)</h3>";

 }

 @GetMapping(value = "/user")

 public String user() {

 return "<h3>Hello User :)</h3>";

 }

 @GetMapping(value = "/")

 public String welcome() {

 return "<h3>Welcome :)</h3>";

 }

}

Step-5) Create Security Configuration class like below with Jdbc Authentication Manager

package in.ashokit;

@Configuration

@EnableWebSecurity

public class SecurityConfiguration {

private static final String ADMIN = "ADMIN";

private static final String USER = "USER";

@Autowired

private DataSource dataSource;

@Autowired

public void authManager(AuthenticationManagerBuilder auth) throws Exception {

auth.jdbcAuthentication()

.dataSource(dataSource)

.passwordEncoder(new BCryptPasswordEncoder())

.usersByUsernameQuery("select username,password,enabled from users where
username=?")

.authoritiesByUsernameQuery("select username,authority from authorities where
username=?");

}

@Bean

public SecurityFilterChain securityConfig(HttpSecurity http) throws Exception {

http.authorizeHttpRequests((req) -> req

.antMatchers("/admin").hasRole(ADMIN)

.antMatchers("/user").hasAnyRole(ADMIN,USER)

.antMatchers("/").permitAll()

.anyRequest().authenticated()

).formLogin();

return http.build();

}

}

=====

OAuth 2.0

=====

1) Create Spring Boot application with below dependencies

<dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-oauth2-client</artifactId>

</dependency>

<dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-security</artifactId>

</dependency>

<dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-web</artifactId>

</dependency>

2) Create OAuth app in Github.com

 (Login --> Settings --> Developer Settings --> OAuth Apps --> Create App --> Copy Client ID & Client Secret)

3) Configure GitHub OAuth App client id & client secret in application.yml file like below

spring:

 security:

 oauth2:

 client:

 registration:

 github:

 clientId:

 clientSecret:

4) Create Rest Controller with method

@RestController

```
public class WelcomeRestController {  
  
    @GetMapping("/")  
    public String welcome() {  
        return "Welcome to Ashok IT";  
    }  
}
```

5) Run the application and test it.

=====

Spring Boot with JWT

=====

-> JWT stands for JSON Web Tokens

-> JSON Web Tokens are an open, industry standard RFC 7519 method for representing claims securely between two parties.

-> JWT official Website : <https://jwt.io/>

-> Below is the sample JWT Token

token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMONTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

-> JWT contains below 3 parts

1) Header

2) Payload

3) Signature

Note: JWT 3 parts will be separated by using dot(.)

=====

1) Create Spring Boot application with below dependencies

=====

```
<dependencies>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-security</artifactId>

    </dependency>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-web</artifactId>

    </dependency>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-devtools</artifactId>

        <scope>runtime</scope>

        <optional>true</optional>

    </dependency>

    <dependency>

        <groupId>org.projectlombok</groupId>

        <artifactId>lombok</artifactId>

        <optional>true</optional>

    </dependency>

    <dependency>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-tomcat</artifactId>

        <scope>provided</scope>

    </dependency>

</dependencies>
```



```
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt</artifactId>
        <version>0.9.1</version>
    </dependency>
    <dependency>
        <groupId>javax.xml.bind</groupId>
        <artifactId>jaxb-api</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
        <exclusions>
            <exclusion>
                <groupId>org.junit.vintage</groupId>
                <artifactId>junit-vintage-engine</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
    <dependency>
        <groupId>org.springframework.security</groupId>
        <artifactId>spring-security-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>
```

```
=====
```

2) Create Request and Response Binding Classes

```
=====
```

@Data

```
public class AuthenticationRequest implements Serializable {  
    private String username;  
    private String password;  
}
```

```
public class AuthenticationResponse implements Serializable {  
    private final String jwt;  
    public AuthenticationResponse(String jwt) {  
        this.jwt = jwt;  
    }  
    public String getJwt() {  
        return jwt;  
    }  
}
```

```
=====
```

3) Create UserDetailsService for credentials configuration

```
=====
```

@Service

```
public class MyUserDetailsService implements UserDetailsService {  
    @Override  
    public UserDetails loadUserByUsername(String s) throws UsernameNotFoundException {  
        return new User("admin",  
"$2a$12$e9oIZjBeSJDryJ/P5p1Ep.WPzJ3f4.C2vHC/as1E22R25XXGpPYyG", new ArrayList<>());  
    }  
}
```

```
=====
```

4) Create JwtUtils class

```
=====
```

@Service

```
public class JwtUtil {

    private String SECRET_KEY = "secret";


    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser().setSigningKey(SECRET_KEY).parseClaimsJws(token).getBody();
    }

    private Boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    public String generateToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        return createToken(claims, userDetails.getUsername());
    }

}
```

```

private String createToken(Map<String, Object> claims, String subject) {

    return Jwts.builder()
        .setClaims(claims)
        .setSubject(subject)
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60 * 10))
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
        .compact();
}

public Boolean validateToken(String token, UserDetails userDetails) {
    final String username = extractUsername(token);
    return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
}
}

```

=====

5) Create Filter class

=====

```

@Component

public class JwtRequestFilter extends OncePerRequestFilter {

    @Autowired
    private MyUserDetailsService userDetailsService;

    @Autowired
    private JwtUtil jwtUtil;
}

```

@Override

```
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain chain)
```

```
throws ServletException, IOException {
```

```
final String authorizationHeader = request.getHeader("Authorization");
```

```
String username = null;
```

```
String jwt = null;
```

```
if (authorizationHeader != null && authorizationHeader.startsWith("Bearer ")) {
```

```
    jwt = authorizationHeader.substring(7);
```

```
    username = jwtUtil.extractUsername(jwt);
```

```
}
```

```
if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
```

```
    UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);
```

```
    if (jwtUtil.validateToken(jwt, userDetails)) {
```

```
        UsernamePasswordAuthenticationToken usernamePasswordAuthenticationToken = new
UsernamePasswordAuthenticationToken(
```

```
            userDetails, null, userDetails.getAuthorities());
```

```
        usernamePasswordAuthenticationToken
```

```
            .setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
```

```
SecurityContextHolder.getContext().setAuthentication(usernamePasswordAuthenticationToken);
```

```
    }
```

```
}
```

```
chain.doFilter(request, response);
```

```
}
```

```
}
```

=====

6) Create WebSecurity Config class

=====

```
package com.ashokit.security;

@Configuration
@EnableWebSecurity

public class WebSecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private UserDetailsService myUserDetailsService;

    @Autowired
    private JwtRequestFilter jwtRequestFilter;

    @Autowired
    public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
        auth.userDetailsService(myUserDetailsService);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Override
    @Bean
    public AuthenticationManager authenticationManagerBean() throws Exception {
        return super.authenticationManagerBean();
    }

    @Override
    protected void configure(HttpSecurity httpSecurity) throws Exception {
        httpSecurity.csrf()

                .disable()

                .authorizeRequests()

                .antMatchers("/authenticate")

                .permitAll()
    }
}
```

```

        .anyRequest()
        .authenticated()
        .and()
        .exceptionHandling()
        .and()
        .sessionManagement()
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS);

```

```

        httpSecurity.addFilterBefore(jwtRequestFilter,
UsernamePasswordAuthenticationFilter.class);
    }
}

```

=====

7) create Rest Controller class

=====

```

package com.ashokit.rest;

@RestController

public class HelloRestController {

    @Autowired

    private AuthenticationManager authenticationManager;

    @Autowired

    private JwtUtil jwtTokenUtil;

    @Autowired

    private MyUserDetailsService userDetailsService;

    @RequestMapping({ "/hello" })

    public String firstPage() {

        return "Hello World";

    }

    @RequestMapping(value = "/authenticate", method =

```

```

RequestMethod.POST)

    public ResponseEntity<?> createAuthenticationToken(@RequestBody AuthenticationRequest
authenticationRequest)

        throws Exception {

        try {

            authenticationManager.authenticate(new
UsernamePasswordAuthenticationToken(

                authenticationRequest.getUsername(),
authenticationRequest.getPassword());

            } catch (BadCredentialsException e) {

                throw new Exception("Incorrect username or password", e);

            }

            final UserDetails userDetails =
userDetailsService.loadUserByUsername(authenticationRequest.getUsername());

            final String jwt = jwtTokenUtil.generateToken(userDetails);

            return ResponseEntity.ok(new AuthenticationResponse(jwt));

        }

    }

```

=====

8) Run the application and Test it

=====

Spring Boot with Redis Cache Integration

Introduction

-> Every application will interact with database to store and retrieve data

-> In DB we will maintain data in tables (Rows & Columns)

-> DB tables are divided into 2 types

1) Transactional Tables

2) Non-Transactional Tables

-> If our application is performing INSERT / UPDATE / DELETE operations in table then it is called as Transactional Table.

-> If our application is performing only SELECT operation in the table then it is called as Non-Transactional table.

Examples :

-> When student register, then our application will insert student details into STUDENT_DTLS table (This is Transactional Table)

-> When student open 'Register For Demo' pop-up it will display COUNTRY Name with Country Code to enter Mobile number. Here our application will read the data from COUNTRY_MASTER table. In our application we don't insert / update / delete data in COUNTRY_MASTER table (This is Non-Transactional table)

-> If we retrieve countries data from table for multiple times also the data will be same and unnecessarily multiple queries will execute and it reduces our application performance.

-> When we have this kind of scenario then its better to go for Cache Memory.

-> Static tables data we should retrieve only one time and store into Cache memory.

-> When our application needs that static we can retrieve from Cache instead of Retrieving from DB.

-> Cache will be created in JVM and our application is also running in same JVM hence retrieval will be faster where as DB calls are costly

=====

What is Cache ?

=====

- > Cache is a memory which is used to store the data in key-value format.
- > If our application having a requirement to use static data then it is highly recommended to use Cache in that application.
- > Using cache data we can reduce no. of db calls and we can improve performance of the application.
- > If we want to use cache in one application then we will go for Local cache. Local cache means we will implement with in our application.
- > If we want to use cache data in several applications then we will go for Global Cache.

=====

How to implement cache

=====

- > Once application got started take data from db and store that into cache in the form of key-value pair.
- > When application needs that data take it from cache memory instead of making db call.
- > By using cache memory we can reduce no. of db calls hence application performance will be improved.

=====

Q) What is Local cache & What is distributed cache ?

=====

- > If we implement cache with in one application then it called as local cache.
- > If we want to use cache data in multiple applications then we should go for distributed cache (Ex: Redis cache)

=====

Q) What is Redis?

=====

- > Redis is an open source cache which is used to store the data in the form key-value pair.
- > Multiple applications can connect to Redis cache to access the data.

=====

Redis Environment Setup

=====

- 1) Download redis-server software
- 2) Extract downloaded zip file
- 3) Double click on redis-server.exe file (redis server will start)
- 4) Double click on redis-cli.exe file
- 5) Type ping in client cmd it will respond with PONG.

Note : With this redis server setup is ready.

Note: In realtime, redis-server will be installed in remote machine and they will share redis-server details (url, uname, pwd and port)

=====

Spring Boot Application with Redis Integration

=====

- 1) Create Spring Boot application with below dependencies

- 1) spring-boot-starter-web
- 2) spring-boot-starter-redis
- 3) project lombok
- 4) devtools
- 5) swagger & swagger-ui

- 2) Configure JedisConnectionFactory bean with Redis Server details & inject JedisConnectionFactory bean into RedisTemplate bean

- 3) Create Binding class class

- 4) Create Service Class with required methods. Inject RedisTemplate into Repository bean and get HashOperations object to perform redis operations.

- 5) Create Rest Controller with required methods

6) Run the application and test it.

=====

Spring Boot with Redis Integration Project Git repo :

https://github.com/ashokitschool/Springboot_Redis_App

=====

=====

Apache Kafka

=====

=> Apache Kafka is a distributed streaming platform

=> Apache Kafka is called as Message Broker

=> Apache Kafka is used to process real time data feeds with high throughput and low latency

Ex : flights data, sensors data, stocks data, news data, social media etc....

=> Kafka works based on Publisher and Subscriber model

=====

Kafka Terminology

=====

Zookeeper

Kafka Server

Kafka Topic

Message

Publisher

Subscriber

=====

Kafka APIs

=====

Connector API

Publisher API

Subscriber API

Streams API

=====

Spring Boot + Apache Kafka Application

=====

Step-1 : Download Zookeeper from below URL

URL : <http://mirrors.estointernet.in/apache/zookeeper/stable/>

Step-2 : Download Apache Kafka from below URL

URL : <http://mirrors.estointernet.in/apache/kafka/>

Step-3 : Set Path to ZOOKEEPER in Environment variables upto bin folder

Note: zookeeper.properties file will be available in kafka/config folder. You can copy zookeeper.properties and server.properties files from kafka/config folder to kafka/bin/windows folder.
###

Step-4 : Start Zookeeper server using below command from kafka/bin/windows folder

Command : `zookeeper-server-start.bat zookeeper.properties`

Step-5: Start Kafka Server using below command from Kafka folder

Command : `kafka-server-start.bat server.properties`

Step-6 : Create Kafka Topic using below command from kafka/bin/windows folder

Command : `kafka-topics.bat --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --topic amazon_orders_topic`

Step-7 : View created Topics using below command

Command : `kafka-topics.bat --list --bootstrap-server localhost:9092`

#####

Kafka Producer App Development

#####

=====

1) Add below dependencies

=====

<dependencies>

 <dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-web</artifactId>

 </dependency>

 <dependency>

 <groupId>org.apache.kafka</groupId>

 <artifactId>kafka-streams</artifactId>

 </dependency>

 <dependency>

 <groupId>org.springframework.kafka</groupId>

 <artifactId>spring-kafka</artifactId>

 </dependency>

 <dependency>

 <groupId>com.fasterxml.jackson.core</groupId>

 <artifactId>jackson-databind</artifactId>

 </dependency>

 <dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-test</artifactId>

 <scope>test</scope>

```
        </dependency>
        <dependency>
            <groupId>org.springframework.kafka</groupId>
            <artifactId>spring-kafka-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>
```

=====

2) Create Kafka Constants class

=====

```
public class AppConstants {

    public static final String TOPIC = "ashokit_order_topic";
    public static final String HOST = "localhost:9092";

}
```

=====

3) Create Model class to represent data

=====

```
@Data
public class Order {
    private String id;
    private Double price;
    private String email;
}
```


=====

4) Create Kafka Producer Config class

=====

@Configuration

public class KafkaProduceConfig {

 @Bean

 public ProducerFactory<String, Order> producerFactory() {

 Map<String, Object> configProps = new HashMap<>();

 configProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
AppConstants.HOST);

 configProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class);

 configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
JsonSerializer.class);

 return new DefaultKafkaProducerFactory<>(configProps);

 }

 @Bean

 public KafkaTemplate<String, Order> kafkaTemplate() {

 return new KafkaTemplate<>(producerFactory());

 }

}

=====

4) Create Service Class

=====

```
@Service

public class OrderService {

    @Autowired
    private KafkaTemplate<String, Order> kafkaTemplate;

    public String addMsg(Order order) {

        // publish msg to kafka topic
        kafkaTemplate.send(AppConstants.TOPIC, order);

        return "Msg Published To Kafka Topic";
    }
}
```

=====

5) Create RestController classs

=====

```
@RestController

public class OrderRestController {

    @Autowired
    private OrderService service;

    @PostMapping("/order")
    public String createOrder(@RequestBody Order order) {

        String msg = service.addMsg(order);

        return msg;
    }
}
```

=====

6) Run the application and test it

=====

```
{  
  "id" : "OD101",  
  "price" : 200.00,  
  "email" : "smith@gmail.com"  
}
```

#####

Kafka Subscriber App Development

#####

=====

1) Add below dependencies

=====

<dependencies>

 <dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-web</artifactId>

 </dependency>

 <dependency>

 <groupId>org.apache.kafka</groupId>

 <artifactId>kafka-streams</artifactId>

 </dependency>

 <dependency>

 <groupId>org.springframework.kafka</groupId>

 <artifactId>spring-kafka</artifactId>

 </dependency>

 <dependency>

```

        <groupId>com.fasterxml.jackson.core</groupId>
        <artifactId>jackson-databind</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.kafka</groupId>
        <artifactId>spring-kafka-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

```

=====

2) Create Constants class

=====

```

public class KafkaConstants {
    public static final String TOPIC = "ashokit_order_topic";
    public static final String HOST = "localhost:9092";
}

```

=====

3) Create Model class

=====

```

@Data
public class Order {
    private String id;
    private Double price;
    private String email;
}

```

=====

4) Create Consumer Config

=====

@Configuration

public class KafkaConsumerConfig {

 @Bean

 public ConsumerFactory<String, Order> consumerFactory() {

 Map<String, Object> configProps = new HashMap<String, Object>();

 configProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
AppConstants.HOST);

 configProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG,
StringDeserializer.class);

 configProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG,
JsonDeserializer.class);

 return new DefaultKafkaConsumerFactory<>(configProps, new StringDeserializer(),
new JsonDeserializer<>());

 }

 @Bean

 public ConcurrentKafkaListenerContainerFactory<String, Order> kafkaListnerFactory() {

 ConcurrentKafkaListenerContainerFactory<String, Order> factory =

 new ConcurrentKafkaListenerContainerFactory<>();

 factory.setConsumerFactory(consumerFactory());

 return factory;

 }

}

=====

5) Add below method in boot app start class

=====

```
@KafkaListener(topics = AppConstants.TOPIC, groupId="group_ashokit_order")
public void subscribeMsg(String order) {
    System.out.print("*** Msg Recieved From Kafka *** :: ");
    System.out.println(order);
    //logic
}
```

=====

6) Run the application

=====

7) Send Request to Producer app and observer Subscriber app console

=====

Spring Batch

=====

=> Spring Batch is a lightweight, comprehensive batch framework designed to enable the development of robust batch applications.

=====

What is Batch Application ?

=====

-> The application which will process bulk of records is called as Batch application.

Ex :

- 1) Sending Monthly Salaries to all employees in company
- 2) Generating Monthly Payslips for employees
- 3) Sending Monthly Bank Account statement to acc holders
- 4) Sending Monthly credit card bill statement to card holders
- 5) Sending Festival Greetings to all customers
- 6) Sending Notices on daily/weekly/monthly/yearly basis...

Process process CSV File =====> Database =====> CSV File

=====

Spring Batch Terminology

=====

- 1) JobLauncher
- 2) Job
- 3) Step
- 4) ItemReader
- 5) ItemProcessor
- 6) ItemWriter
- 7) JobRepository

=====

Spring Boot with Batch Example

=====

Requirement : Read Customers Data From CSV file and write into Database table

1) Create Spring Starter project with below dependencies

a) web-starter

b) batch

c) data-jpa

d) mysql-driver

e) lombok

2) Configure Datasource properties in application.yml file

3) Keep Source File under src/main/resources folder (customers.csv)

4) Create Entity class & Repository interface

5) Create Batch Configuration class

6) Create RestController and launch job

Git Hub Repo URL : https://github.com/ashokitschool/springboot_batch_app.git

=====

Reactive Programming

=====

- 1) Synchronus based execution (Blocking Thread)
- 2) Asynchronus based execution (Non Blocking Thread)

=> Spring 5.x introduced Reactive Programming

=> In Spring 5.x 'starter-webflux' introduced

=====

Old Approach

=====

@RestController

public class WelcomeRestController{

 @GetMapping("/msg")

 public String getMsg(){

 return "Hello";

 }

}

=====

New Approach

=====

@Component

public class MessageRequestHandler{

 public Mono<ServerResponse> handle(ServletRequest request){

 return new ServerRespoinse.ok()

 .contentType(MediaType.APPLICATION_JSON)

 .body(BodyInserters.fromValue(data));

 }

}

```

@Configuration
public MsgRouter {

    @Bean
    public RouterFunction<ServerResponse> route(MessageRequestHandler requestHandler)
    {

        return RouterFunctions.route(GET("/hello"))

        .and(accept(MediaType.APPLICATION_JSON), MessageRequestHandler::handle);

    }

}

```

=====

SpringBoot Reactive Example

=====

1) Create Boot application with 'Reactive Web' dependency

```

<dependency>

    <groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-webflux</artifactId>

</dependency>

```

Note: Reactive Web dependency means 'starter-webflux' dependency. It will provide 'Netty' as default embedded container.

2) Create Binding class to response

```

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Greeting {

    private String msg;

}

```

3) Create Request Handler class like below

@Component

public class GreetingHandler {

public Mono<ServerResponse> hello(ServerRequest request){

return ServerResponse.ok()

.contentType(MediaType.APPLICATION_JSON)

.body(BodyInserters.fromValue(new Greeting("Hello
World"))));

}

}

4) Create Router class

@Configuration

public class GreetingRouter {

@Bean

public RouterFunction<ServerResponse> route(GreetingHandler greeting){

return RouterFunctions

.route(GET("/hello"))

.and(accept(MediaType.APPLICATION_JSON)), greeting::hello);

}

}

5) Run the application and test it.

=====

ELK Stack

=====

=> ELK is the combination of 3 open source products

1) Elastic Search : It is used to store and process logs

2) Logstash : It is used to collect application logs and store in Elastic Search

3) Kibana : It will provide user interface to monitor application logs

=> By using the above 3 products we can implement Log Aggregation and Logs Monitoring

=====

ELK Setup

=====

1) Download ELK Softwares

=> Elastic Search : <https://www.elastic.co/downloads/elasticsearch>

=> Kibana : <https://www.elastic.co/downloads/kibana>

=> Logstash : <https://www.elastic.co/downloads/logstash>

2) Extract all zip files

3) Run `elasticsearch` using `elasticsearch.bat` file (make sure all security settings disable in `elasticsearch.yml` before running)

\$ `elasticsearch.bat`

4) Check Elastic Search Running or not (URL : <http://localhost:9200/>)

5) Run kibana using kibana.bat file (before running kibana, enable elasticsearch url in kibana.yml file)

```
$ kibana.bat
```

6) Check Kibana running or not (URL : <http://localhost:5601/app/home>)

7) Run Spring Boot Application and generate log file with log messages

8) create logstash.conf file like below

Sample Logstash configuration for creating a simple

Beats -> Logstash -> Elasticsearch pipeline.

```
input {
  file {
    path => "C:/Users/ashok/classes/22-JRTP/workspace/SpringBoot_REST_API/app.log"
    start_position => "beginning"
  }
}

output {
  elasticsearch {
    hosts => ["http://localhost:9200"]
  }
}
```

9) Run logstash server using below command

```
$ logstash -f logstash-sample.conf
```

10) Check logstash server is running or not (<http://localhost:9600>)

11) Check application logs in Kibana dashboard

